Mathematics of Machine Learning

Symmetry



Inductive Bias, Symmetry, Invariance, Group theory

Outline

1. **Inductive Bias**
2. Invariance
3. Group Theory
4. Symmetry in Machine Learning



Inductive Bias

- Learning machines differ in how they split up the space of inputs
- Perceptrons split the space using a hyperplane, RBFs split the space depending on some set of centres, MLPs successively split the world at different layers
- These differences are known as the **inductive bias** of the machine
- The different inductive biases lead to different performance
- How can we exploit the inductive bias to get good performance?

Inductive Bias

- Learning machines differ in how they split up the space of inputs
- Perceptrons split the space using a hyperplane, RBFs split the space depending on some set of centres, MLPs successively split the world at different layers
- These differences are known as the **inductive bias** of the machine
- The different inductive biases lead to different performance
- How can we exploit the inductive bias to get good performance?

Inductive Bias

- Learning machines differ in how they split up the space of inputs
- Perceptrons split the space using a hyperplane, RBFs split the space depending on some set of centres, MLPs successively split the world at different layers
- These differences are known as the **inductive bias** of the machine
- The different inductive biases lead to different performance
- How can we exploit the inductive bias to get good performance?

Inductive Bias

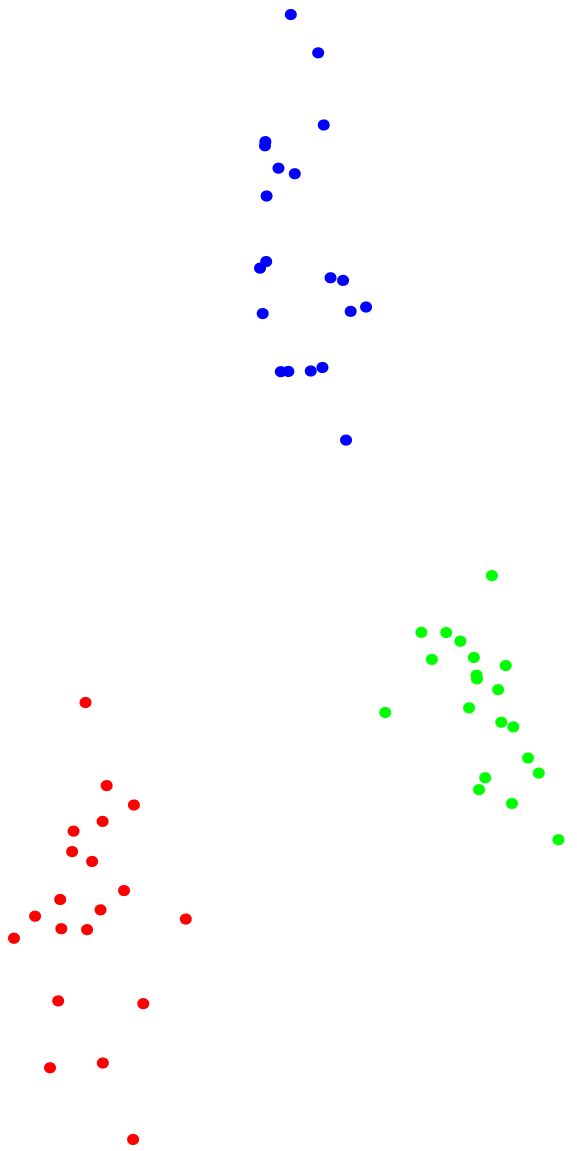
- Learning machines differ in how they split up the space of inputs
- Perceptrons split the space using a hyperplane, RBFs split the space depending on some set of centres, MLPs successively split the world at different layers
- These differences are known as the **inductive bias** of the machine
- The different inductive biases lead to different performance
- How can we exploit the inductive bias to get good performance?

Inductive Bias

- Learning machines differ in how they split up the space of inputs
- Perceptrons split the space using a hyperplane, RBFs split the space depending on some set of centres, MLPs successively split the world at different layers
- These differences are known as the **inductive bias** of the machine
- The different inductive biases lead to different performance
- How can we exploit the inductive bias to get good performance?

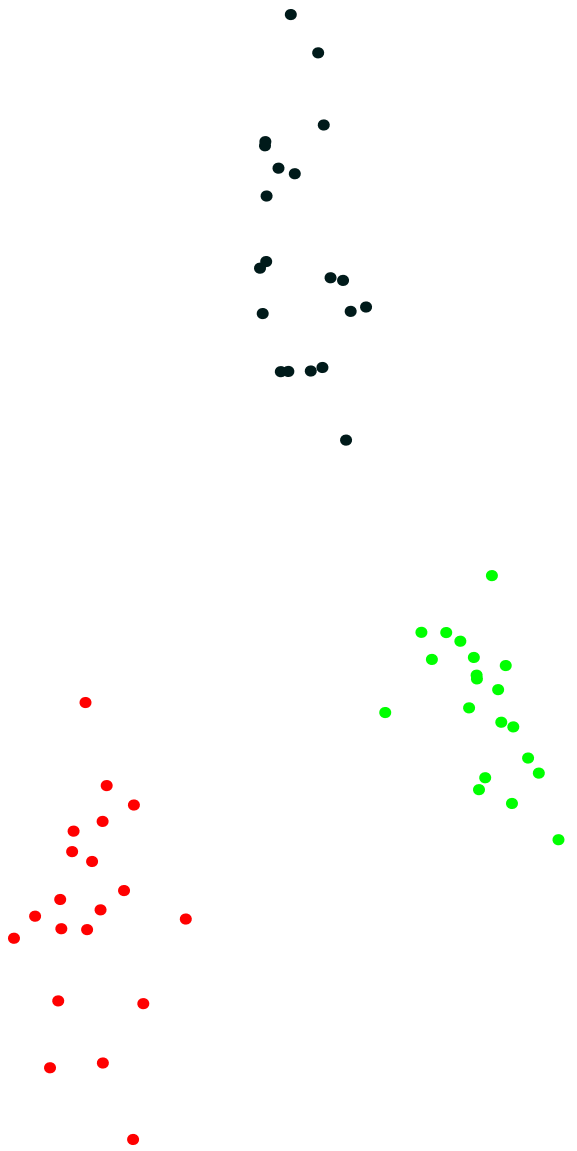
The Real World

- The problems that we care about come from a highly structured world where the solutions inherit some of the properties of that world
- Usually similar input have similar outputs (we don't expect to change one pixel in an image and that it should be classified differently)
- The classes are often quite well separated



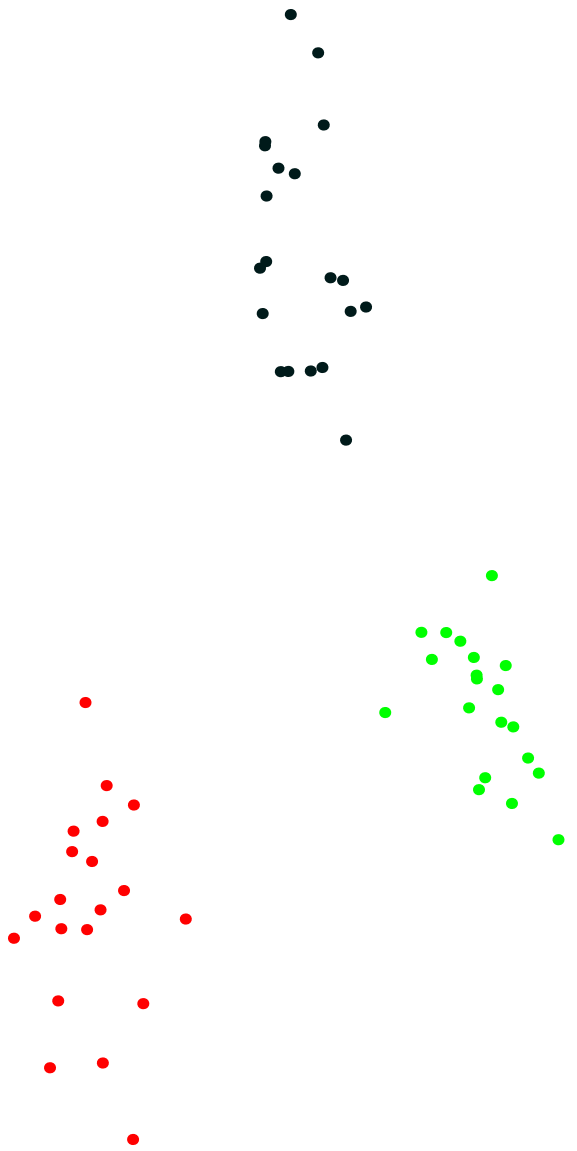
The Real World

- The problems that we care about come from a highly structured world where the solutions inherit some of the properties of that world
- Usually similar input have similar outputs (we don't expect to change one pixel in an image and that it should be classified differently)
- The classes are often quite well separated



The Real World

- The problems that we care about come from a highly structured world where the solutions inherit some of the properties of that world
- Usually similar input have similar outputs (we don't expect to change one pixel in an image and that it should be classified differently)
- The classes are often quite well separated



Smoothness

- A natural assumption is smoothness (the classification surface or regression surface changes slowly as we change the input)
- Often there is a hyperparameter that controls this
  - ⋆ The kernel parameter γ in SVMs (or ℓ in Gaussian Processes)
  - ⋆ k in k-nearest neighbours
  - ⋆ The size of the L_2 regularisation term in neural networks
- We often have to choose these using trial and error (using a validation set)

Smoothness

- A natural assumption is smoothness (the classification surface or regression surface changes slowly as we change the input)
- Often there is a hyperparameter that controls this
  - ⋆ The kernel parameter γ in SVMs (or ℓ in Gaussian Processes)
  - ⋆ k in k-nearest neighbours
  - ⋆ The size of the L_2 regularisation term in neural networks
- We often have to choose these using trial and error (using a validation set)

Smoothness

- A natural assumption is smoothness (the classification surface or regression surface changes slowly as we change the input)
- Often there is a hyperparameter that controls this
  - ⋆ The kernel parameter γ in SVMs (or ℓ in Gaussian Processes)
  - ⋆ k in k-nearest neighbours
  - ⋆ The size of the L_2 regularisation term in neural networks
- We often have to choose these using trial and error (using a validation set)

Outline

1. Inductive Bias
2. **Invariance**
3. Group Theory
4. Symmetry in Machine Learning



Symmetry

- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit

Symmetry

- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit

Symmetry

- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit

Symmetry

- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit

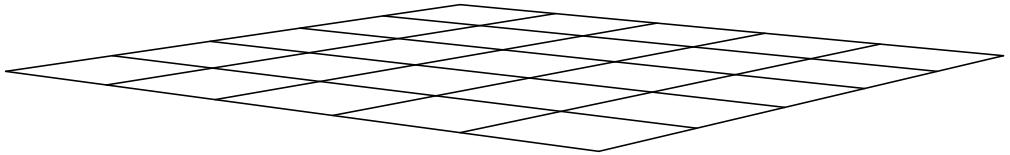
Symmetry

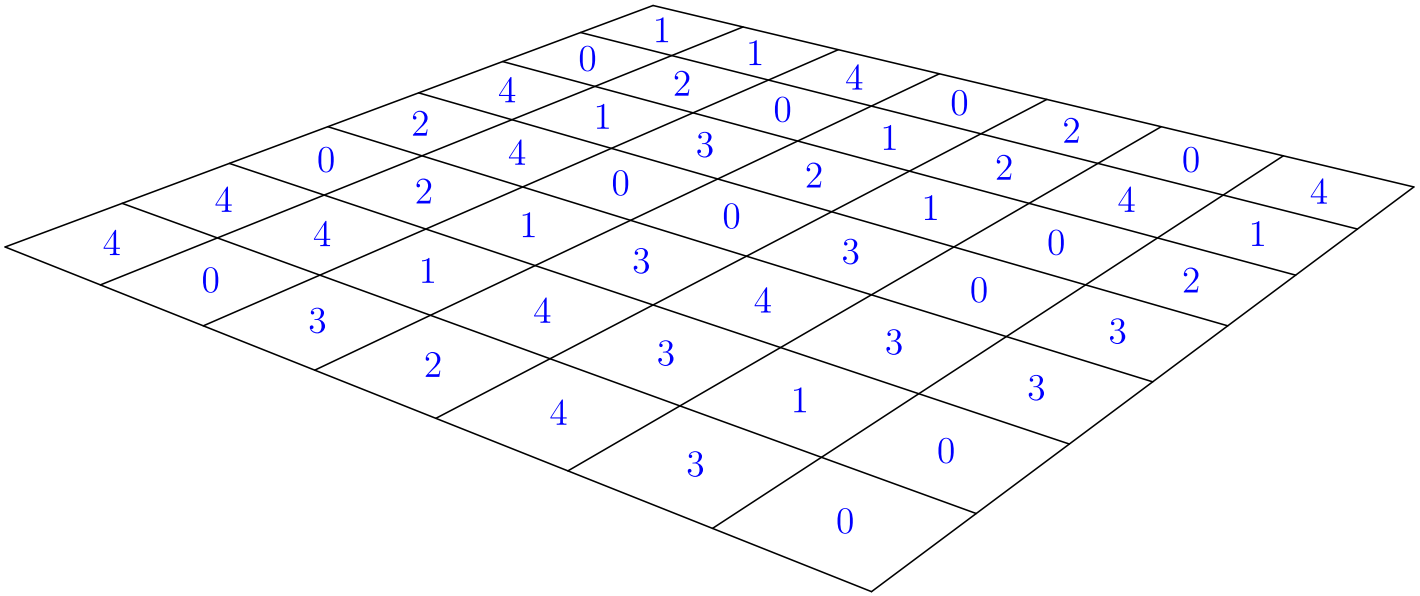
- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit

Symmetry

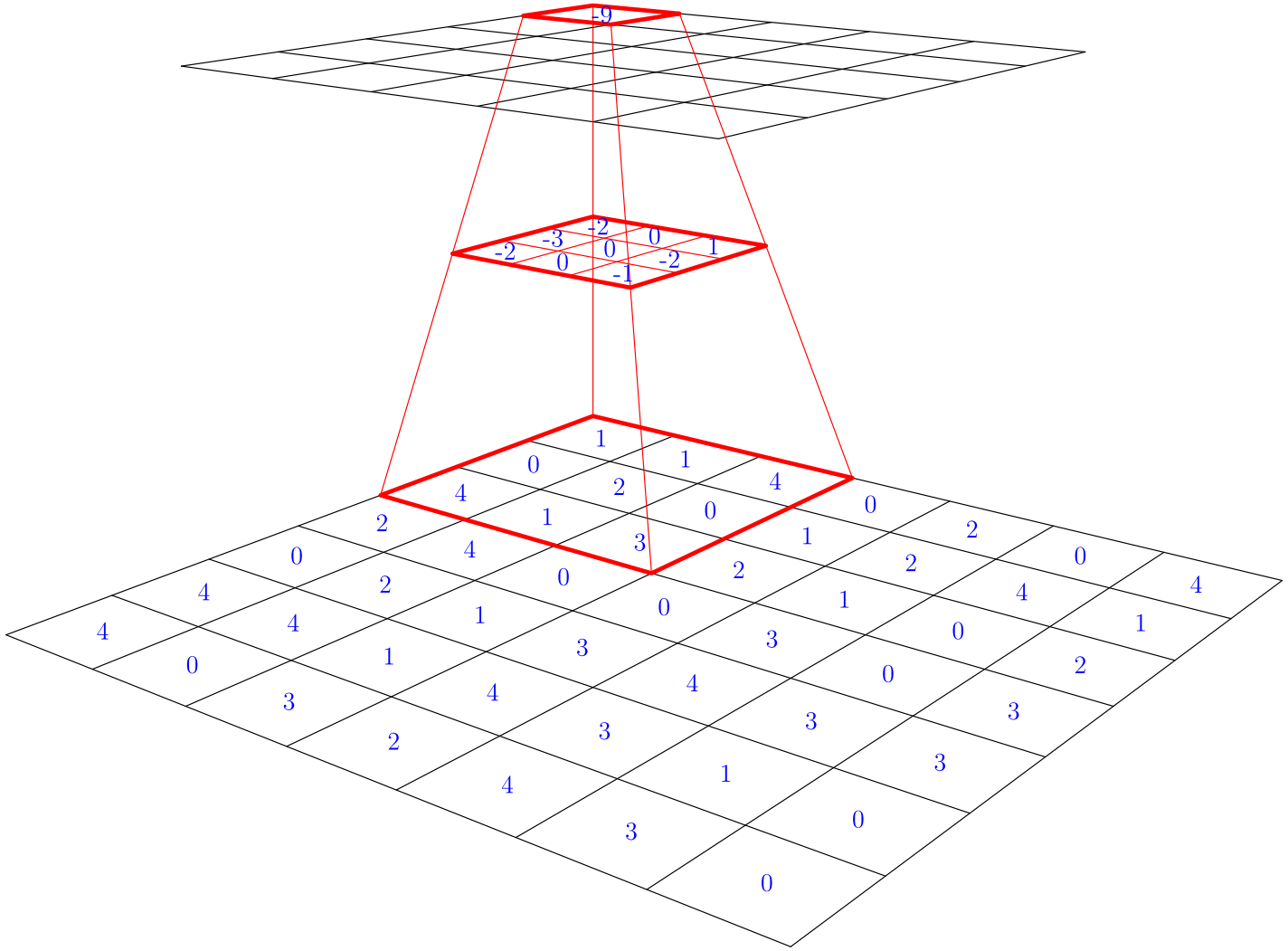
- There are sometimes symmetries in the world we can exploit
- In image classification we consider an object should be classified the same wherever it is in the image
- The classification should be invariant to translations (scaling and perhaps rotations)
- The classification function possesses a translational symmetry
- Learning machines that respect translational invariance have far less propensity to overfit—they don't need to learn that the same object in a different part of the image is the same

Convolutional Neural Networks



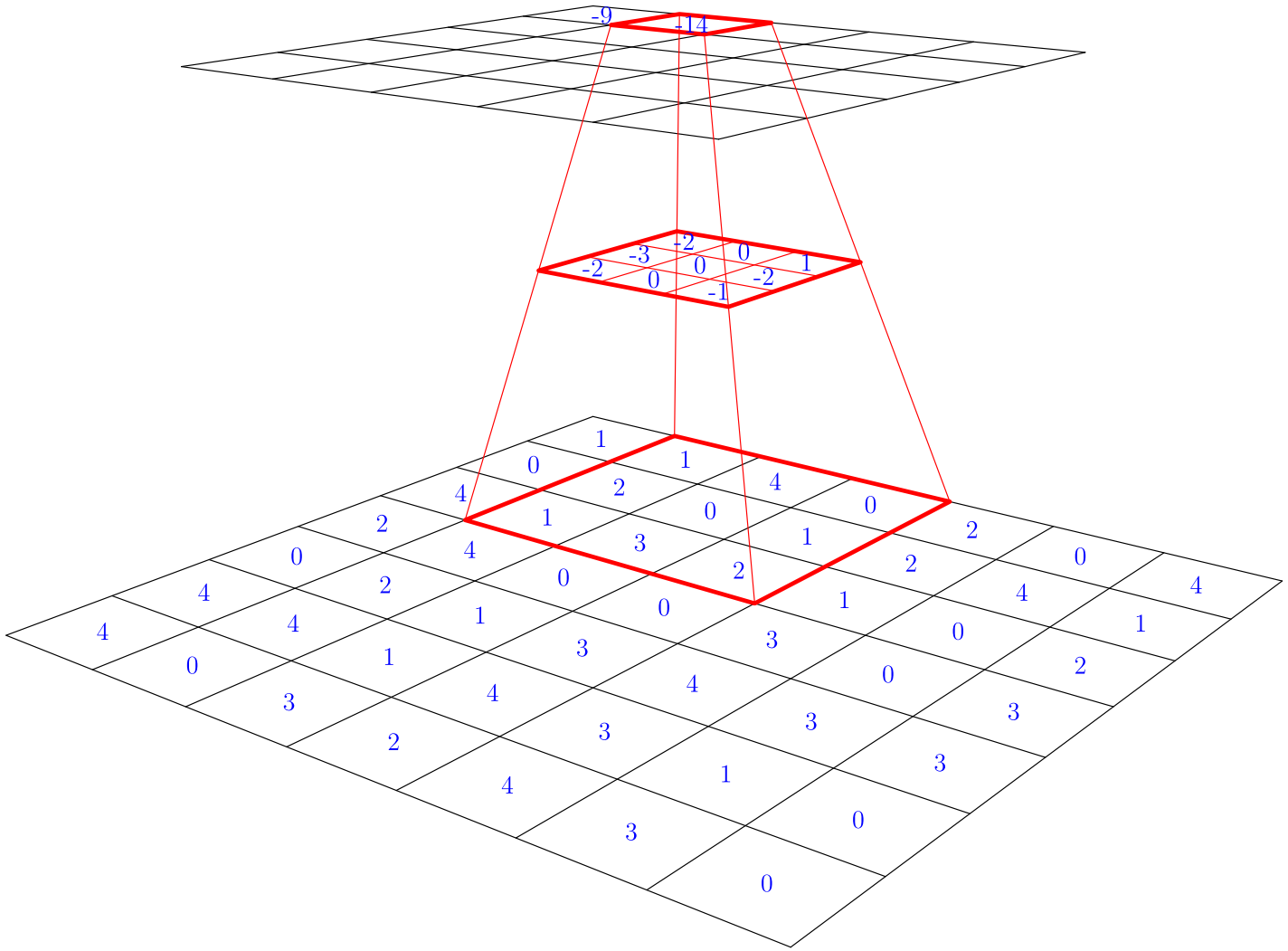


Convolutional Neural Networks



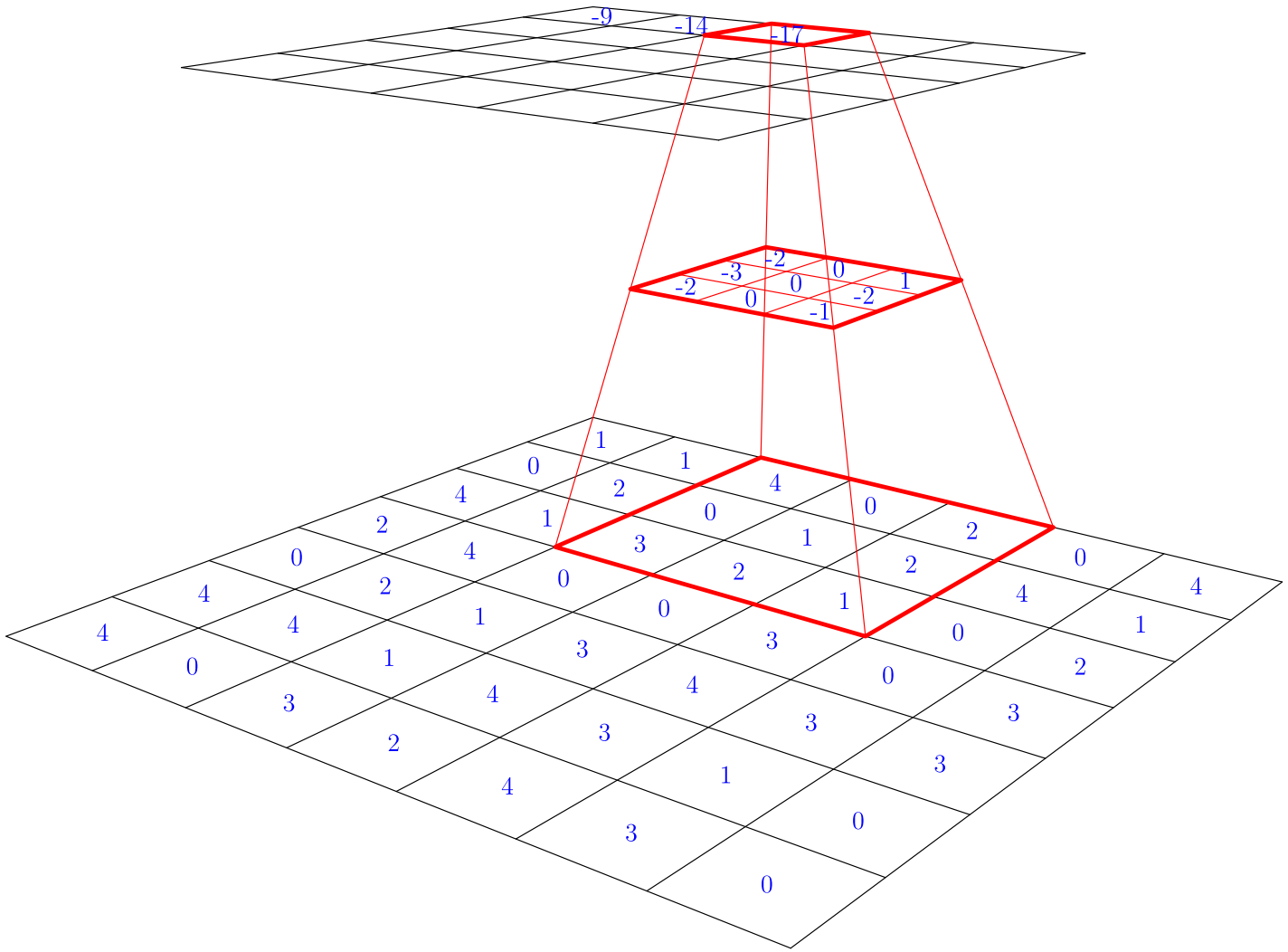
$-2 \times 1 + 0 \times 1 + 1 \times 4$
$-3 \times 0 + 0 \times 2 - 2 \times 0$
$-2 \times 4 + 0 \times 1 - 1 \times 3 = -9$

Convolutional Neural Networks



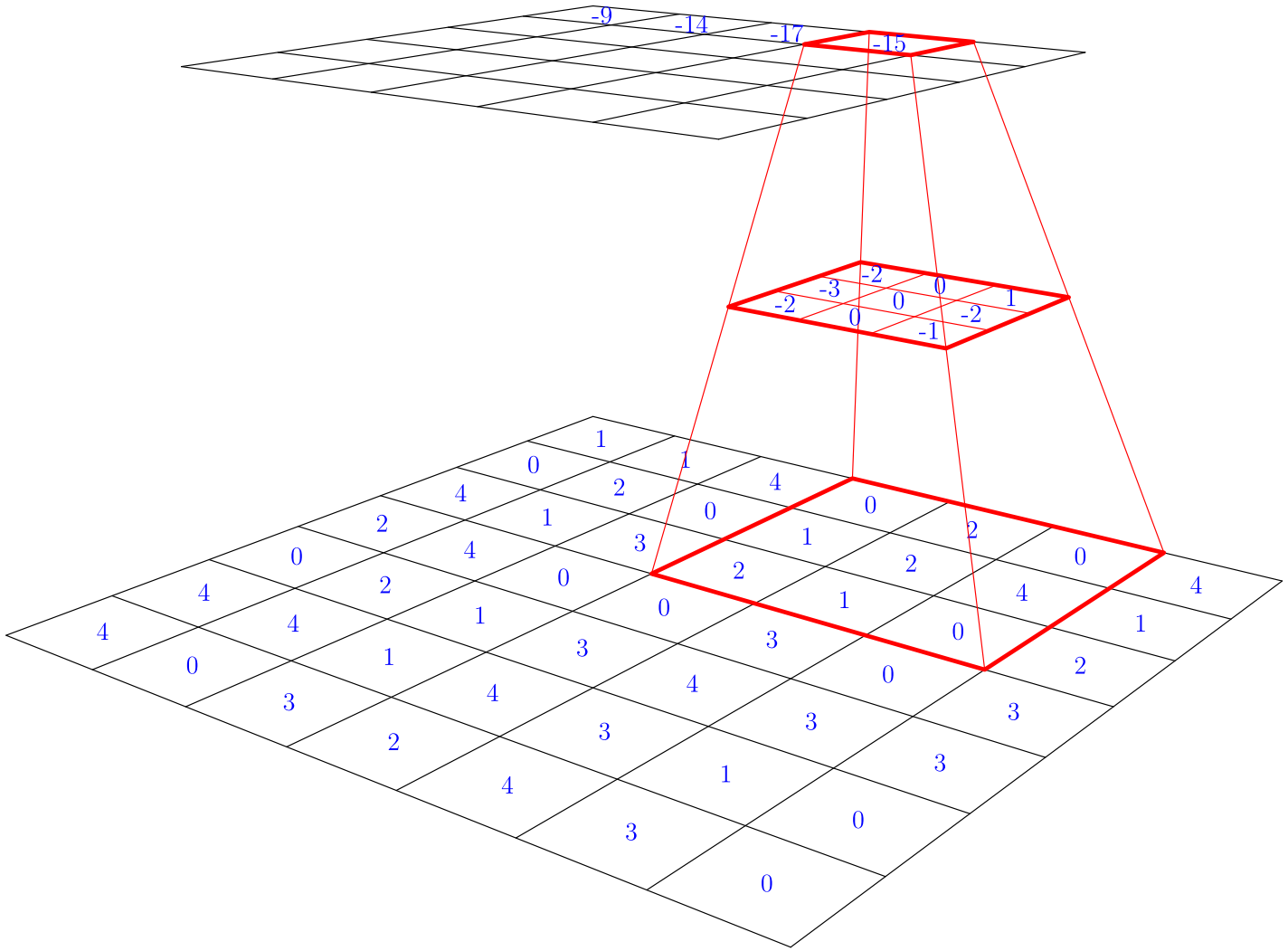
$$-2 \times 1 + 0 \times 4 + 1 \times 0$$
$$-3 \times 2 + 0 \times 0 - 2 \times 1$$
$$-2 \times 1 + 0 \times 3 - 1 \times 2 = -14$$

Convolutional Neural Networks



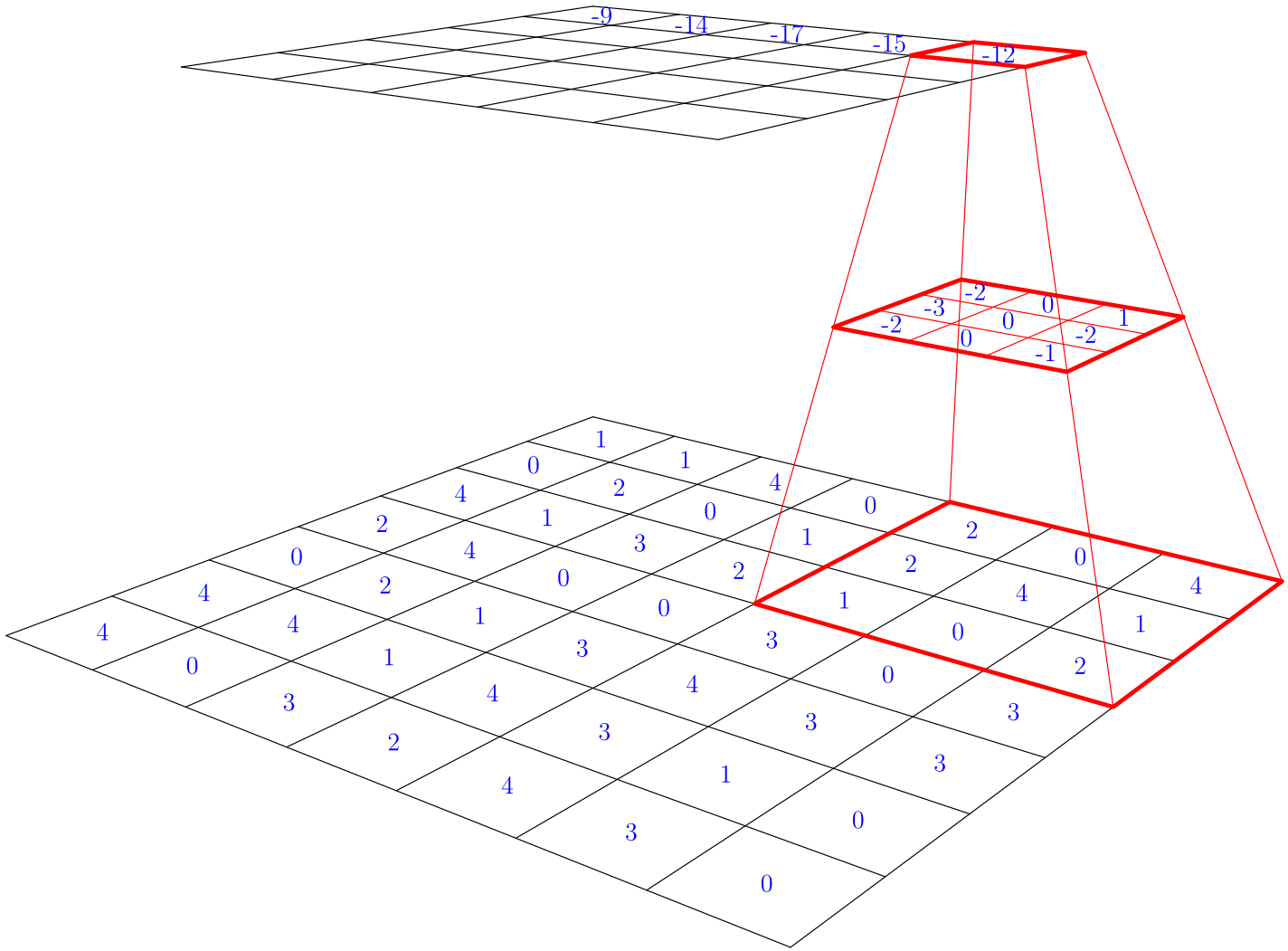
$-2 \times 4 + 0 \times 0 + 1 \times 2$
$-3 \times 0 + 0 \times 1 - 2 \times 2$
$-2 \times 3 + 0 \times 2 - 1 \times 1 = -17$

Convolutional Neural Networks



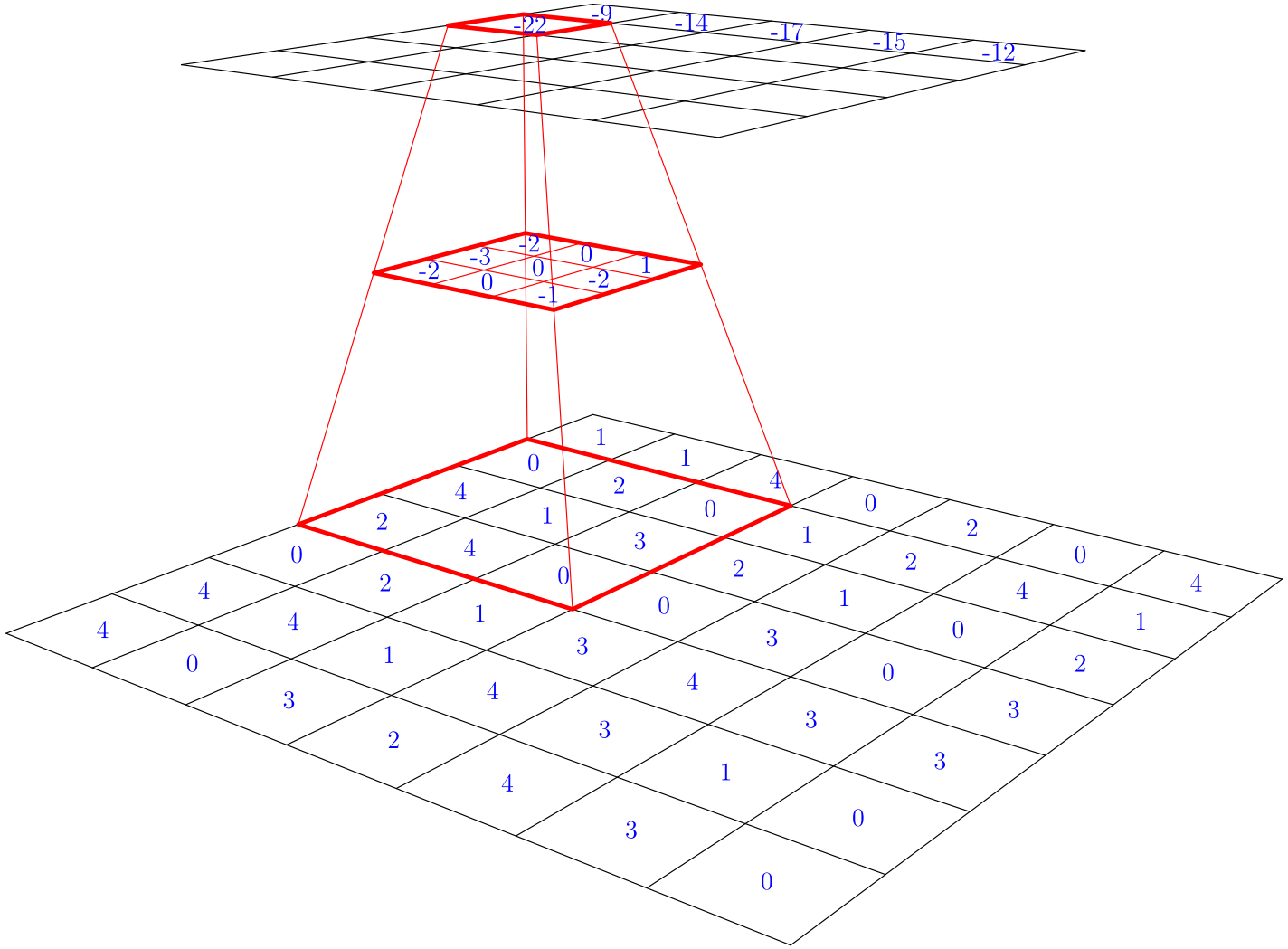
$$-2 \times 0 + 0 \times 2 + 1 \times 0$$
$$-3 \times 1 + 0 \times 2 - 2 \times 4$$
$$-2 \times 2 + 0 \times 1 - 1 \times 0 = -15$$

Convolutional Neural Networks



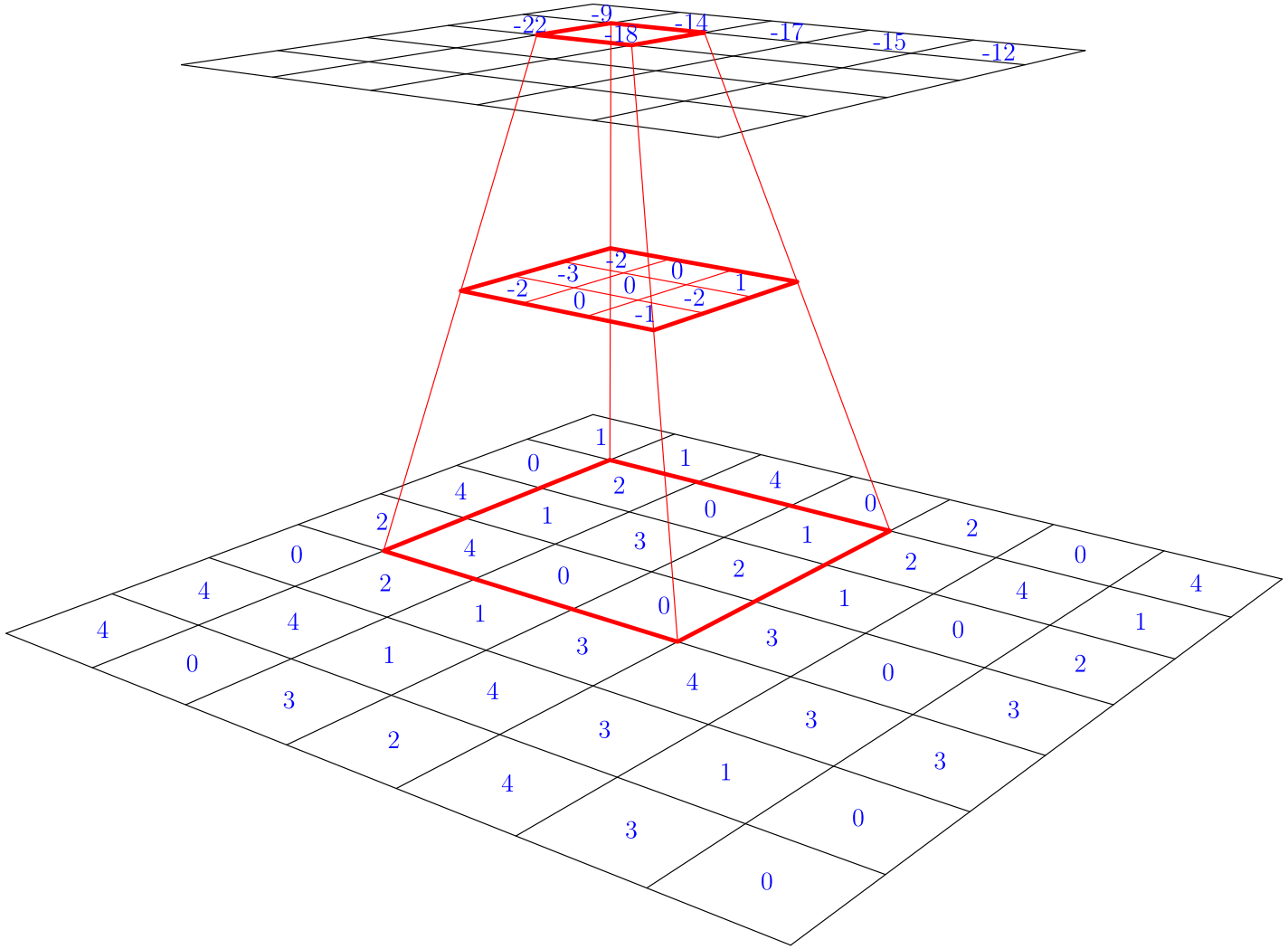
$$-2 \times 2 + 0 \times 0 + 1 \times 4$$
$$-3 \times 2 + 0 \times 4 - 2 \times 1$$
$$-2 \times 1 + 0 \times 0 - 1 \times 2 = -12$$

Convolutional Neural Networks



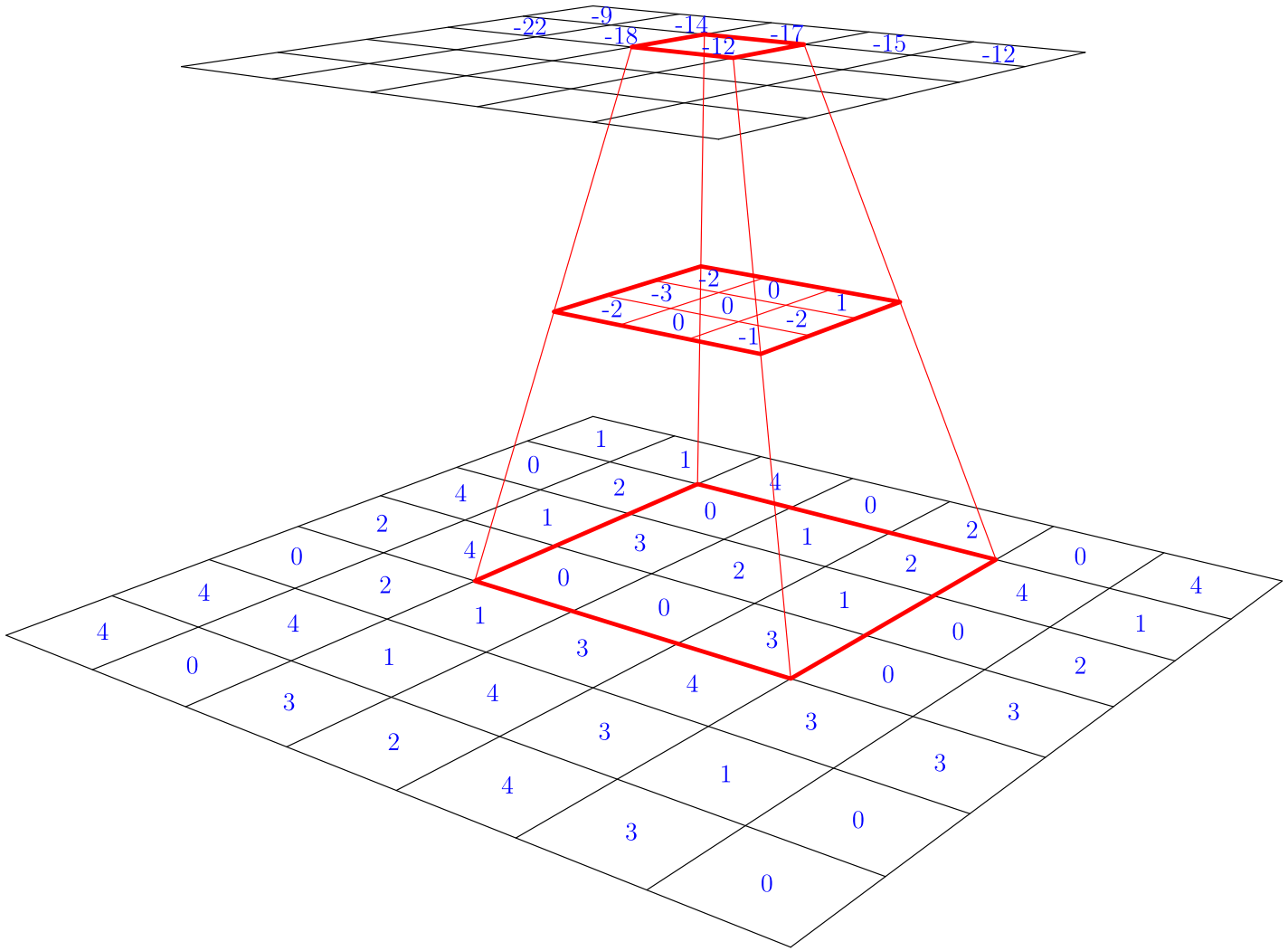
$$-2 \times 0 + 0 \times 2 + 1 \times 0$$
$$-3 \times 4 + 0 \times 1 - 2 \times 3$$
$$-2 \times 2 + 0 \times 4 - 1 \times 0 = -22$$

Convolutional Neural Networks



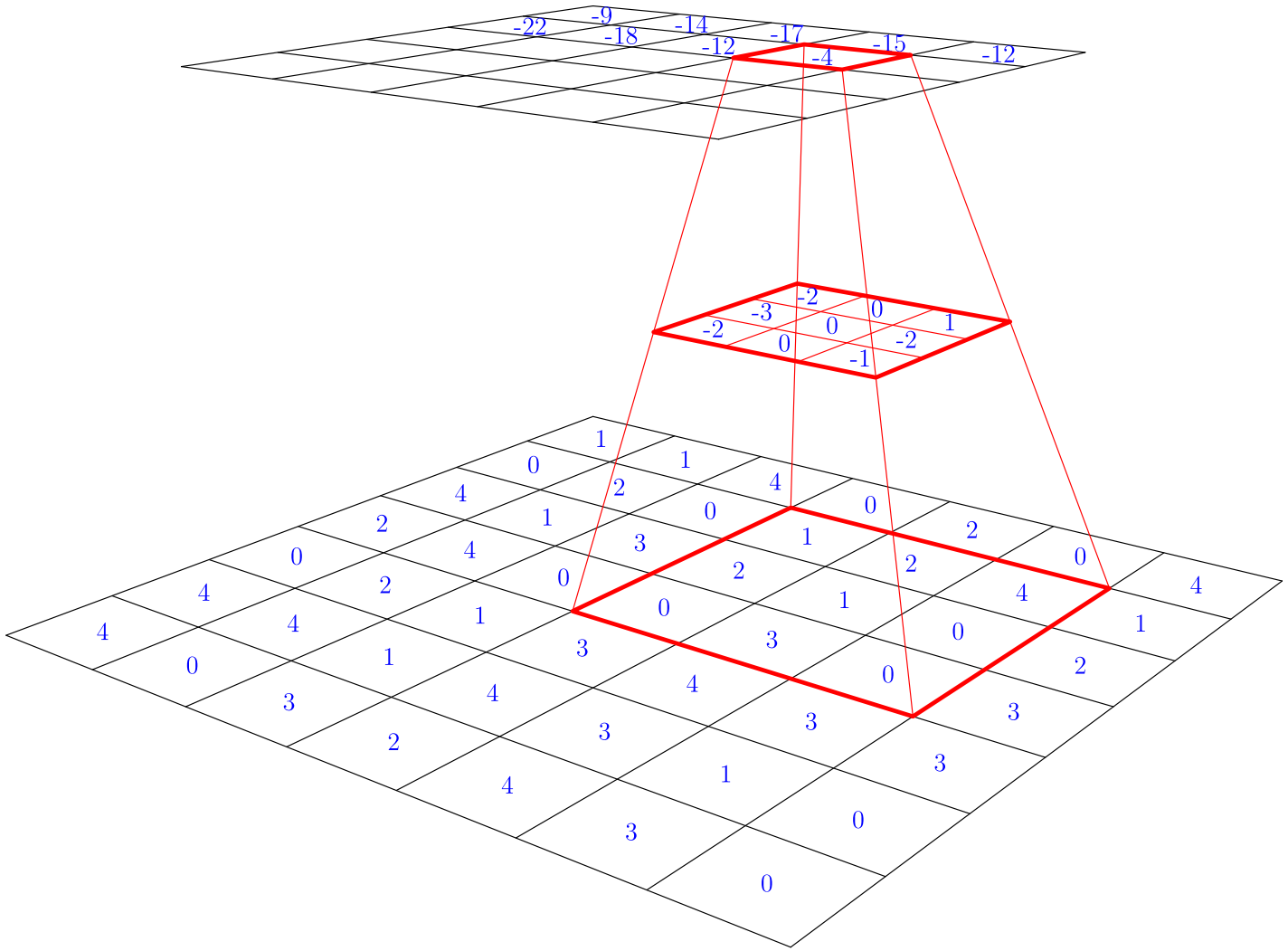
$-2 \times 2 + 0 \times 0 + 1 \times 1$
$-3 \times 1 + 0 \times 3 - 2 \times 2$
$-2 \times 4 + 0 \times 0 - 1 \times 0 = -18$

Convolutional Neural Networks



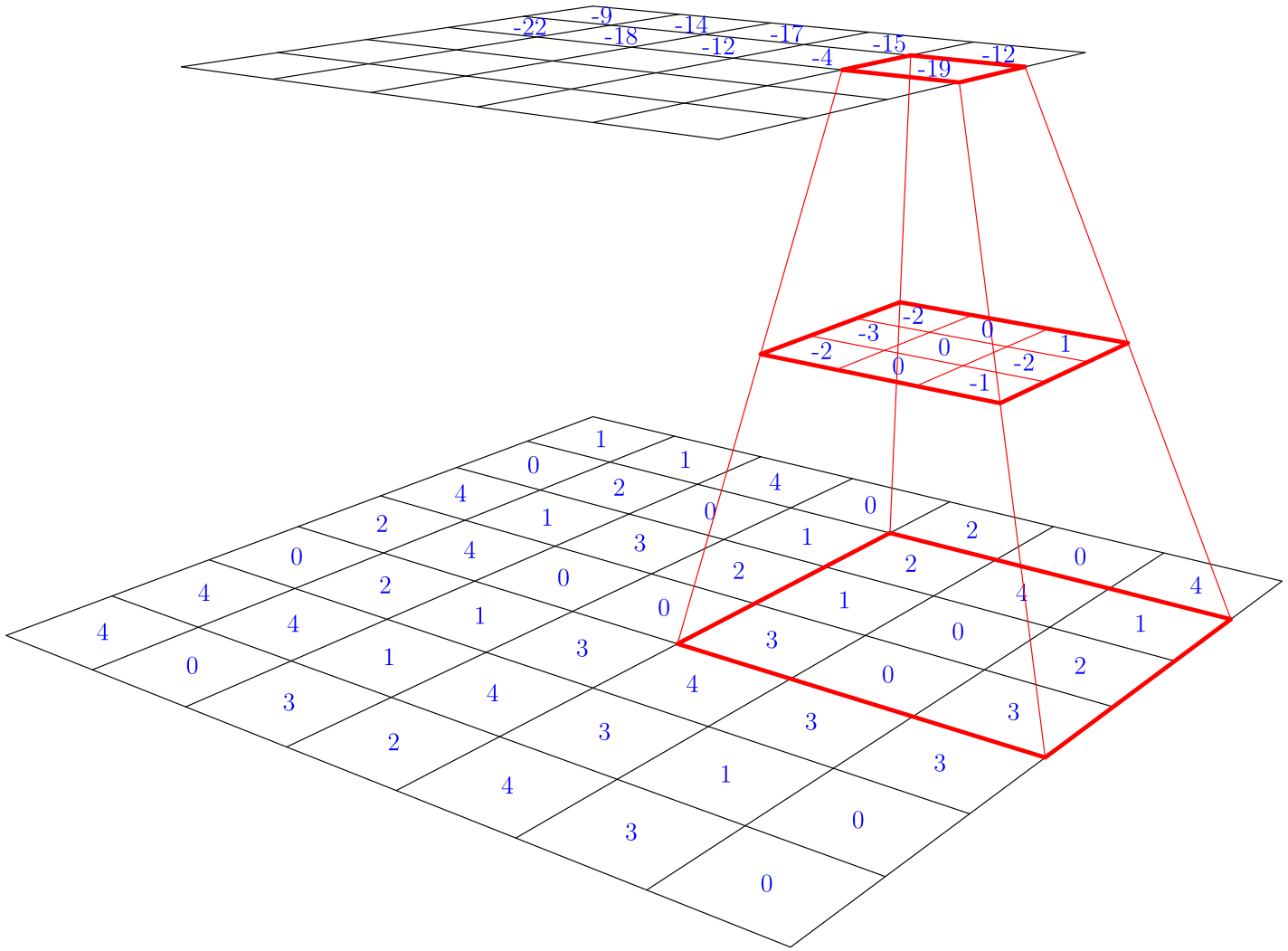
$$-2 \times 0 + 0 \times 1 + 1 \times 2$$
$$-3 \times 3 + 0 \times 2 - 2 \times 1$$
$$-2 \times 0 + 0 \times 0 - 1 \times 3 = -12$$

Convolutional Neural Networks



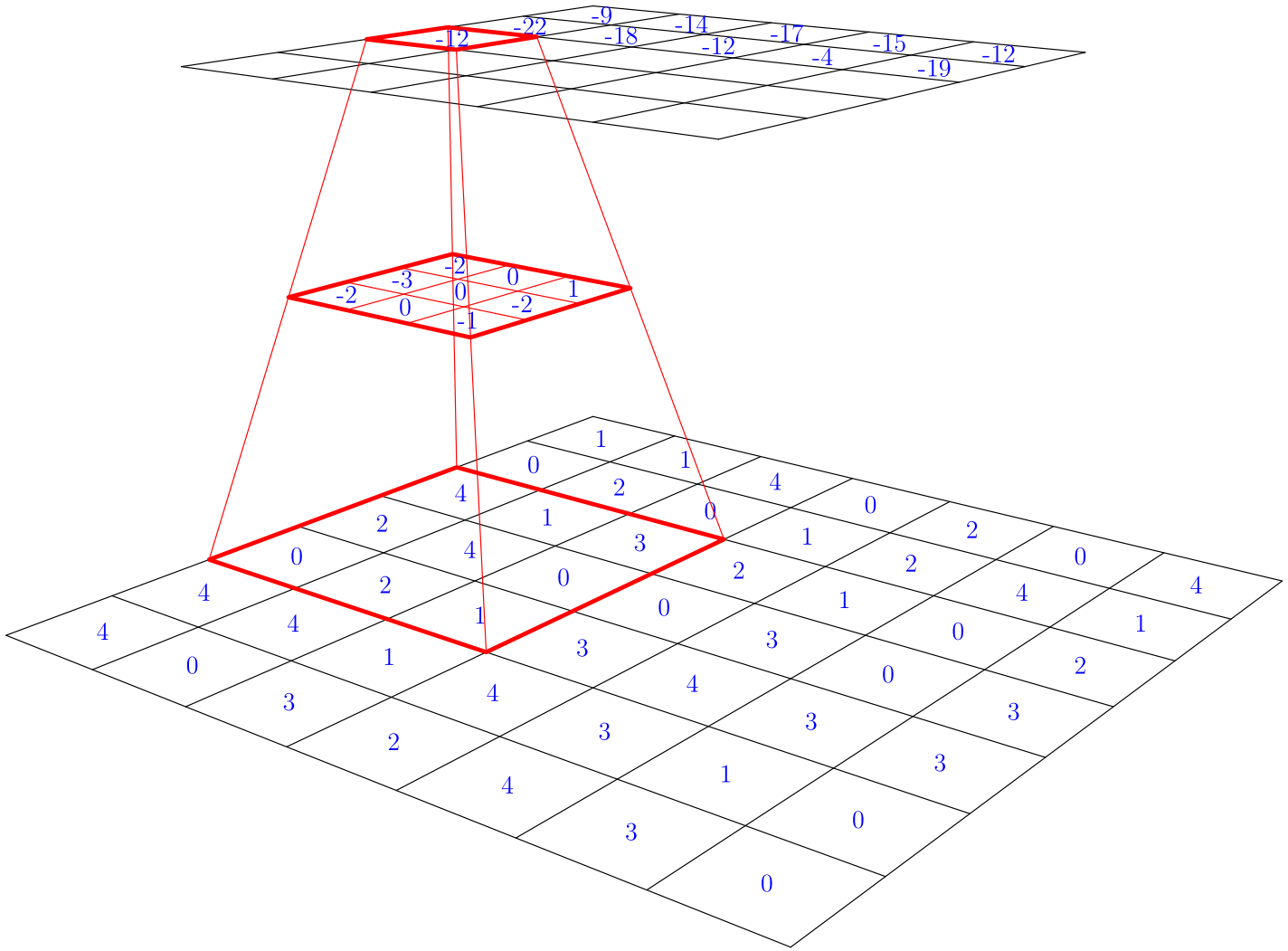
$$-2 \times 1 + 0 \times 2 + 1 \times 4$$
$$-3 \times 2 + 0 \times 1 - 2 \times 0$$
$$-2 \times 0 + 0 \times 3 - 1 \times 0 = -4$$

Convolutional Neural Networks



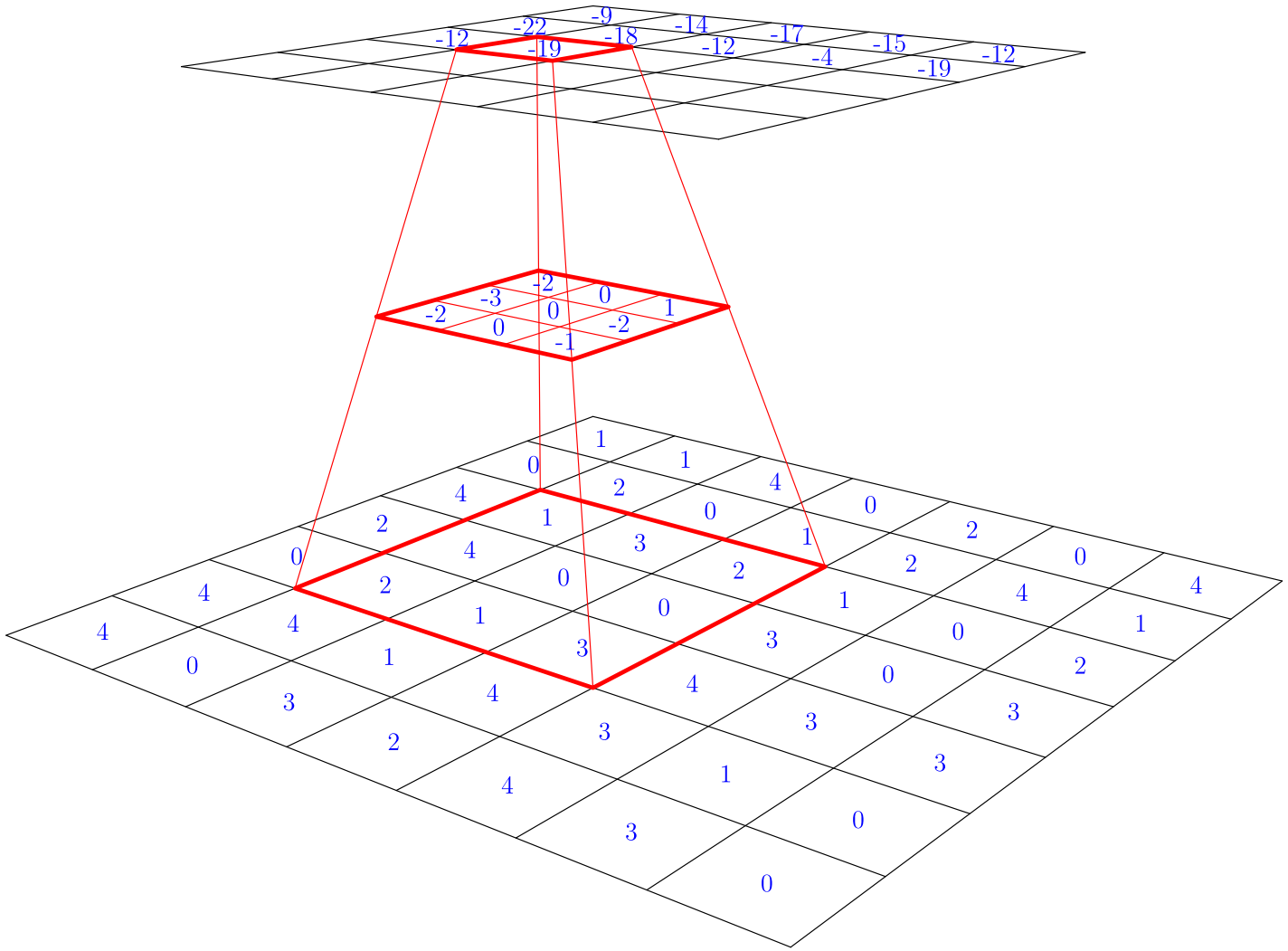
$$-2 \times 2 + 0 \times 4 + 1 \times 1$$
$$-3 \times 1 + 0 \times 0 - 2 \times 2$$
$$-2 \times 3 + 0 \times 0 - 1 \times 3 = -19$$

Convolutional Neural Networks



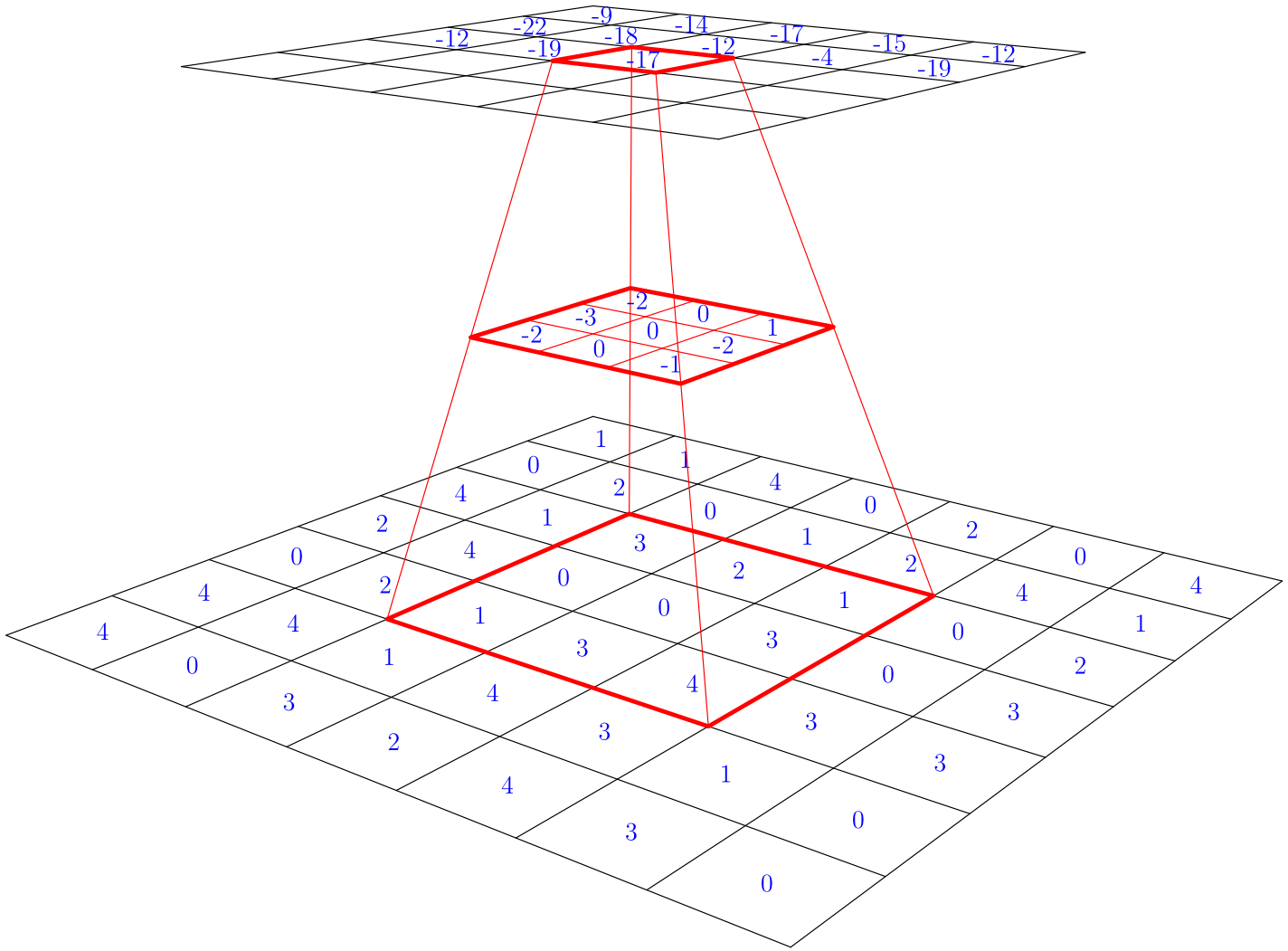
$$-2 \times 4 + 0 \times 1 + 1 \times 3$$
$$-3 \times 2 + 0 \times 4 - 2 \times 0$$
$$-2 \times 0 + 0 \times 2 - 1 \times 1 = -12$$

Convolutional Neural Networks



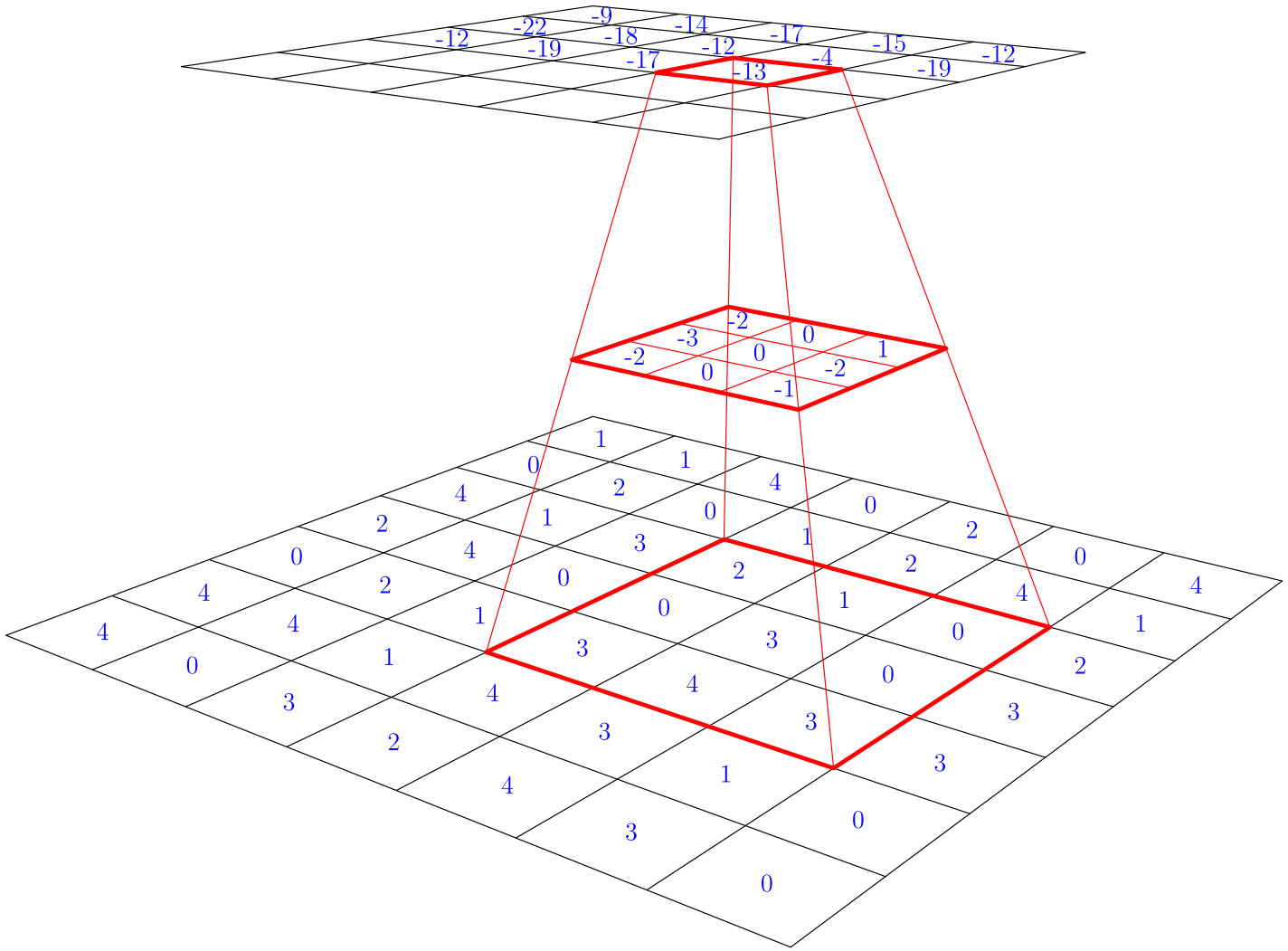
$$-2 \times 1 + 0 \times 3 + 1 \times 2$$
$$-3 \times 4 + 0 \times 0 - 2 \times 0$$
$$-2 \times 2 + 0 \times 1 - 1 \times 3 = -19$$

Convolutional Neural Networks



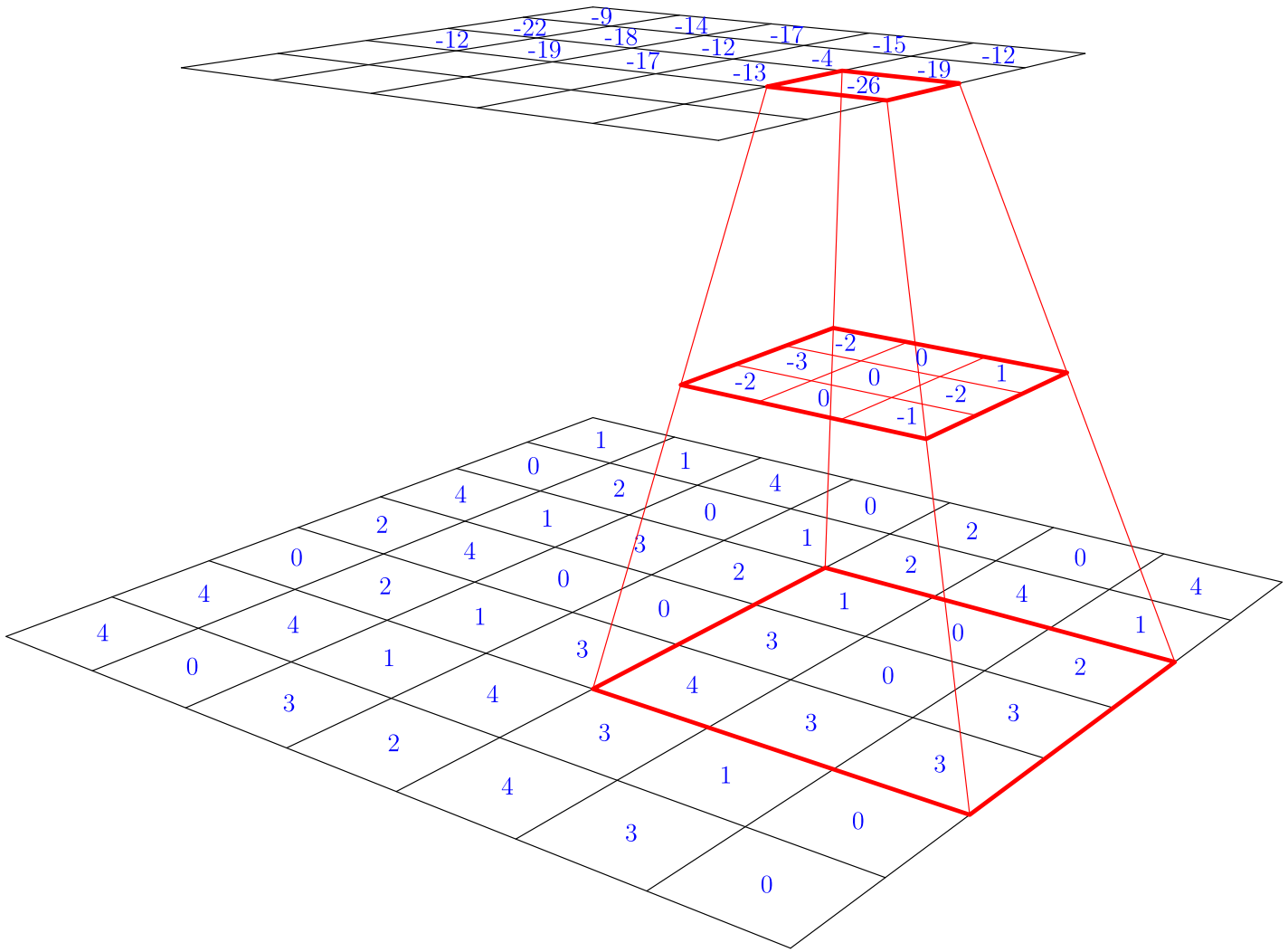
$$-2 \times 3 + 0 \times 2 + 1 \times 1$$
$$-3 \times 0 + 0 \times 0 - 2 \times 3$$
$$-2 \times 1 + 0 \times 3 - 1 \times 4 = -17$$

Convolutional Neural Networks



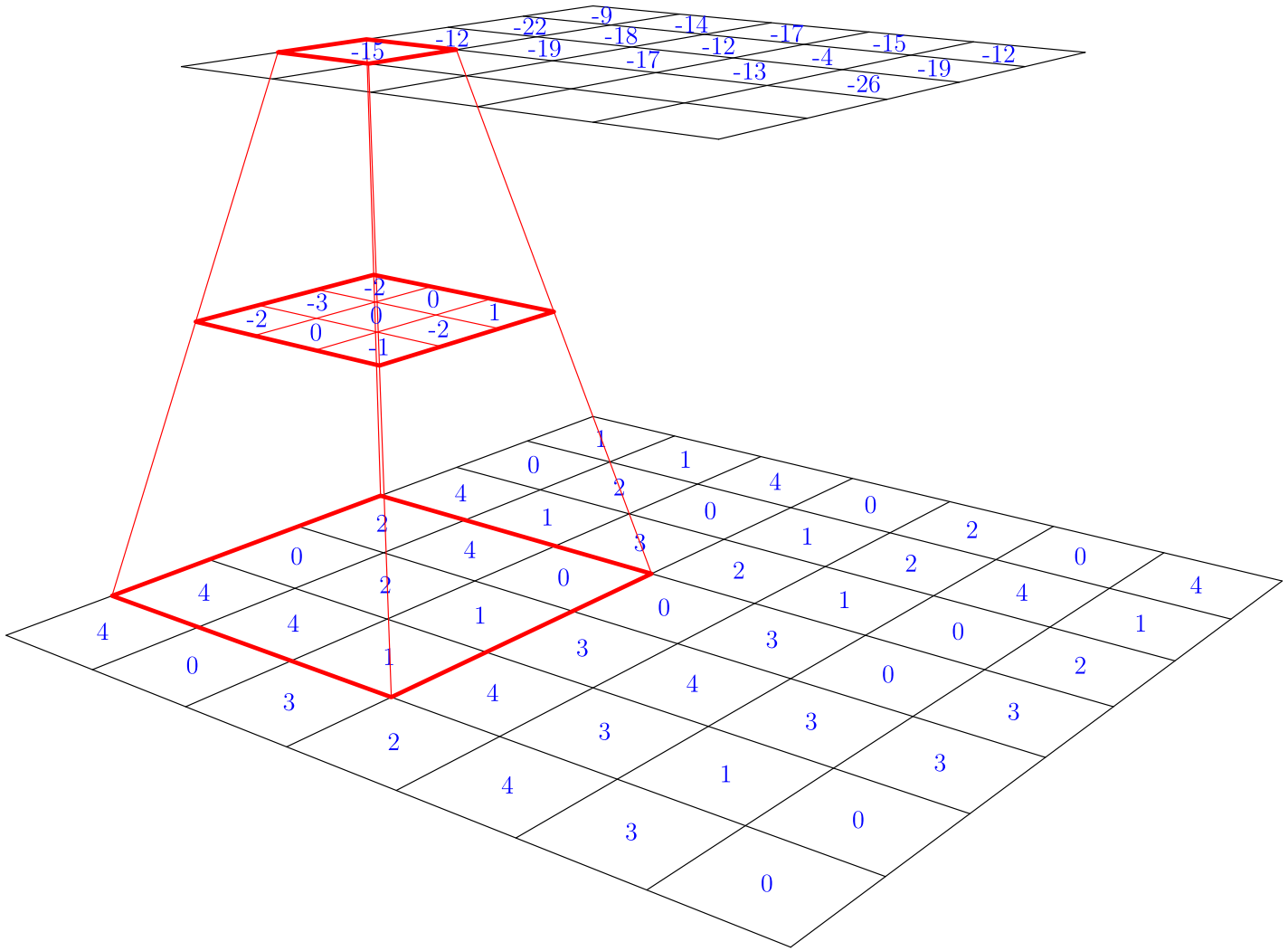
$$-2 \times 2 + 0 \times 1 + 1 \times 0$$
$$-3 \times 0 + 0 \times 3 - 2 \times 0$$
$$-2 \times 3 + 0 \times 4 - 1 \times 3 = -13$$

Convolutional Neural Networks



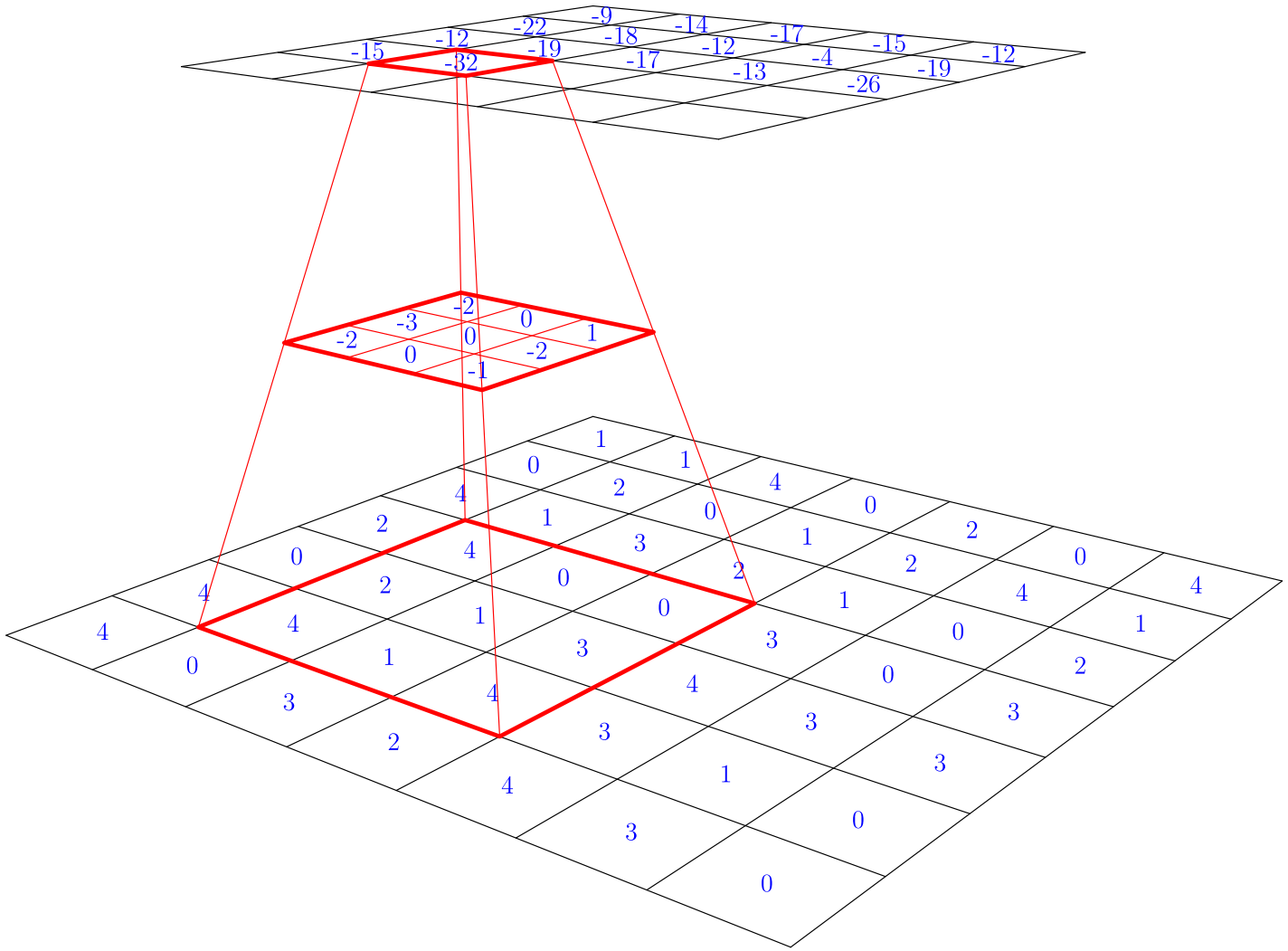
$-2 \times 1 + 0 \times 0 + 1 \times 2$
$-3 \times 3 + 0 \times 0 - 2 \times 3$
$-2 \times 4 + 0 \times 3 - 1 \times 3 = -26$

Convolutional Neural Networks



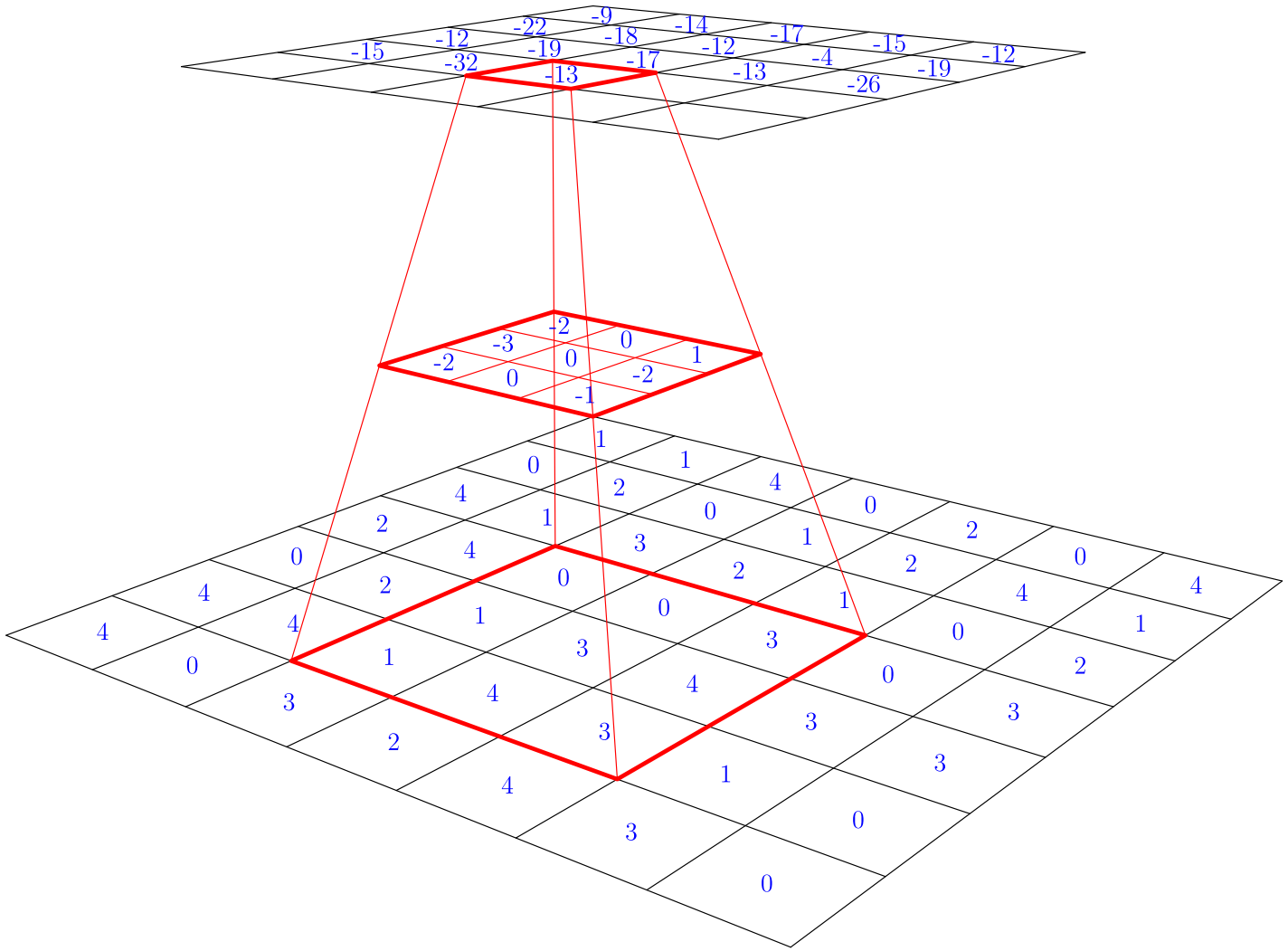
$$-2 \times 2 + 0 \times 4 + 1 \times 0$$
$$-3 \times 0 + 0 \times 2 - 2 \times 1$$
$$-2 \times 4 + 0 \times 4 - 1 \times 1 = -15$$

Convolutional Neural Networks



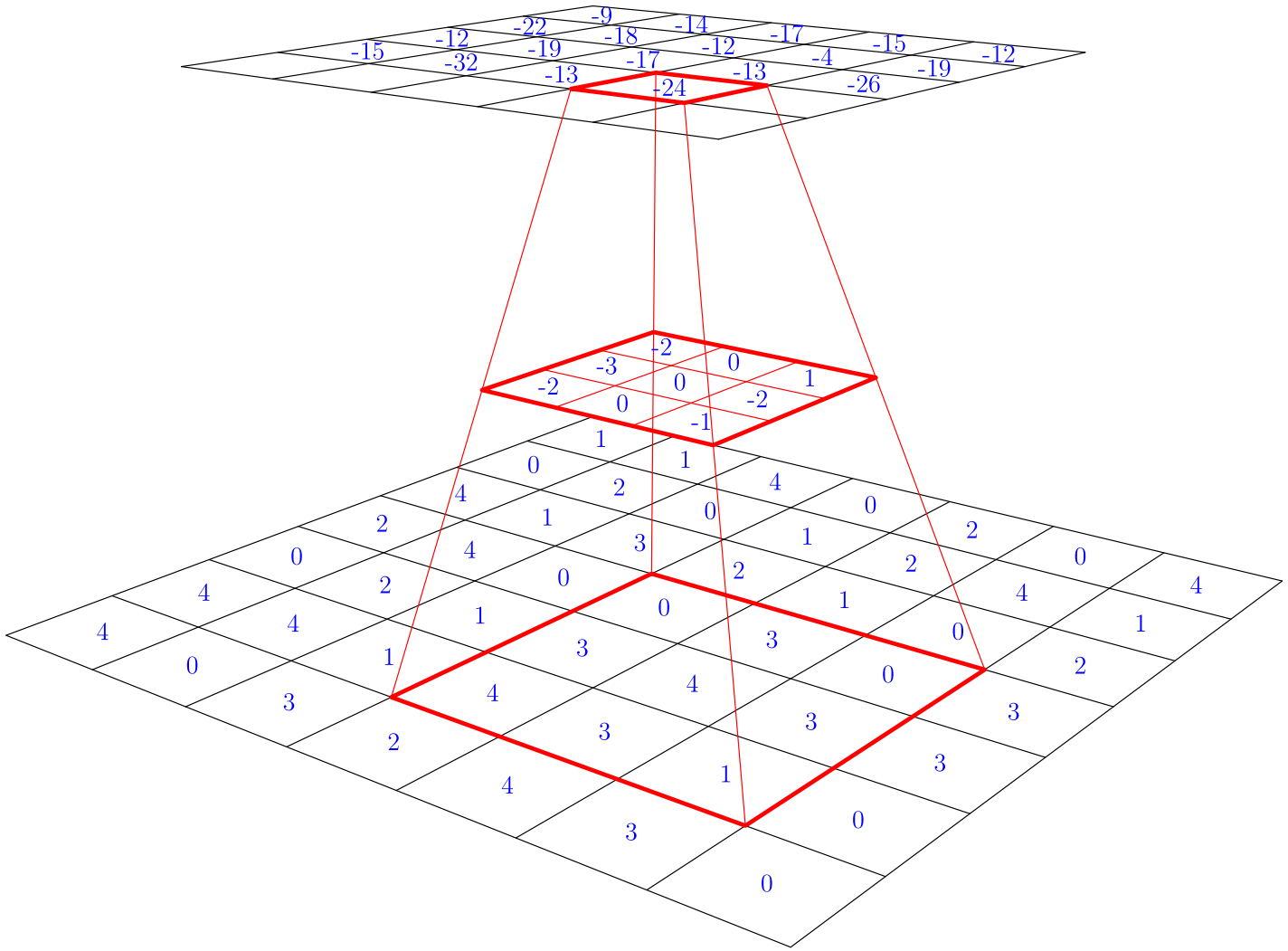
$-2 \times 4 + 0 \times 0 + 1 \times 0$
$-3 \times 2 + 0 \times 1 - 2 \times 3$
$-2 \times 4 + 0 \times 1 - 1 \times 4 = -32$

Convolutional Neural Networks



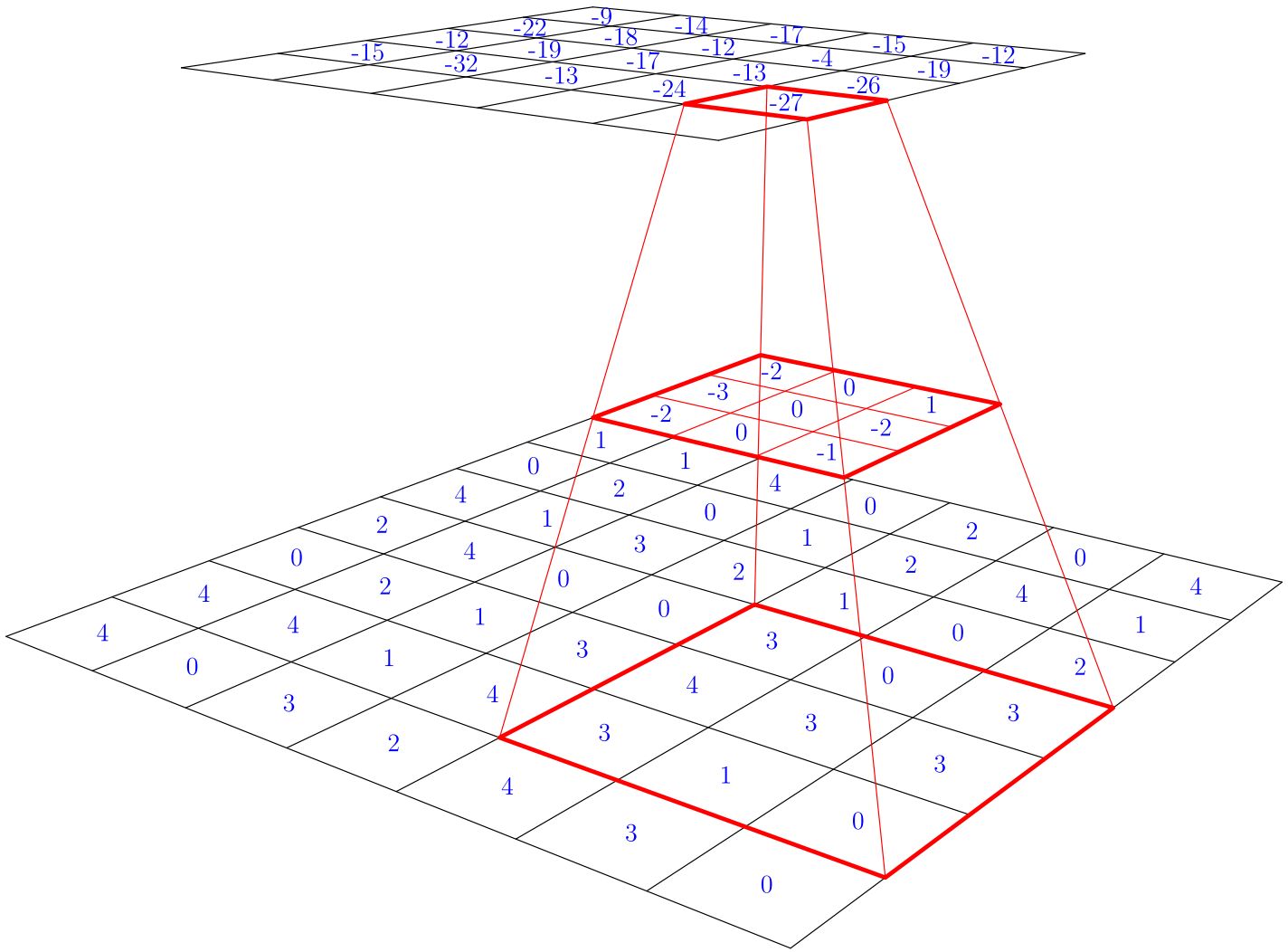
$-2 \times 0 + 0 \times 0 + 1 \times 3$
$-3 \times 1 + 0 \times 3 - 2 \times 4$
$-2 \times 1 + 0 \times 4 - 1 \times 3 = -13$

Convolutional Neural Networks



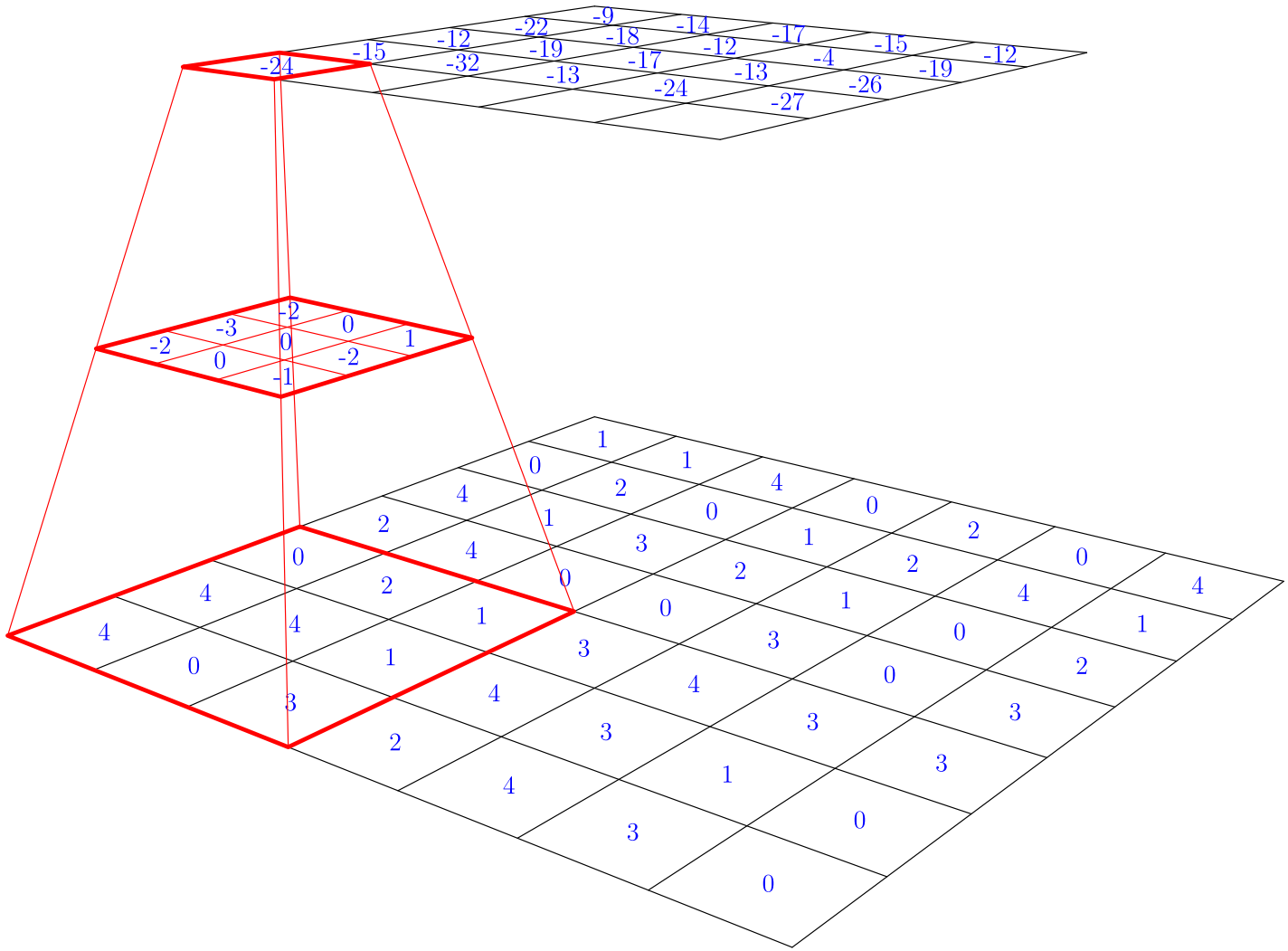
$$-2 \times 0 + 0 \times 3 + 1 \times 0$$
$$-3 \times 3 + 0 \times 4 - 2 \times 3$$
$$-2 \times 4 + 0 \times 3 - 1 \times 1 = -24$$

Convolutional Neural Networks



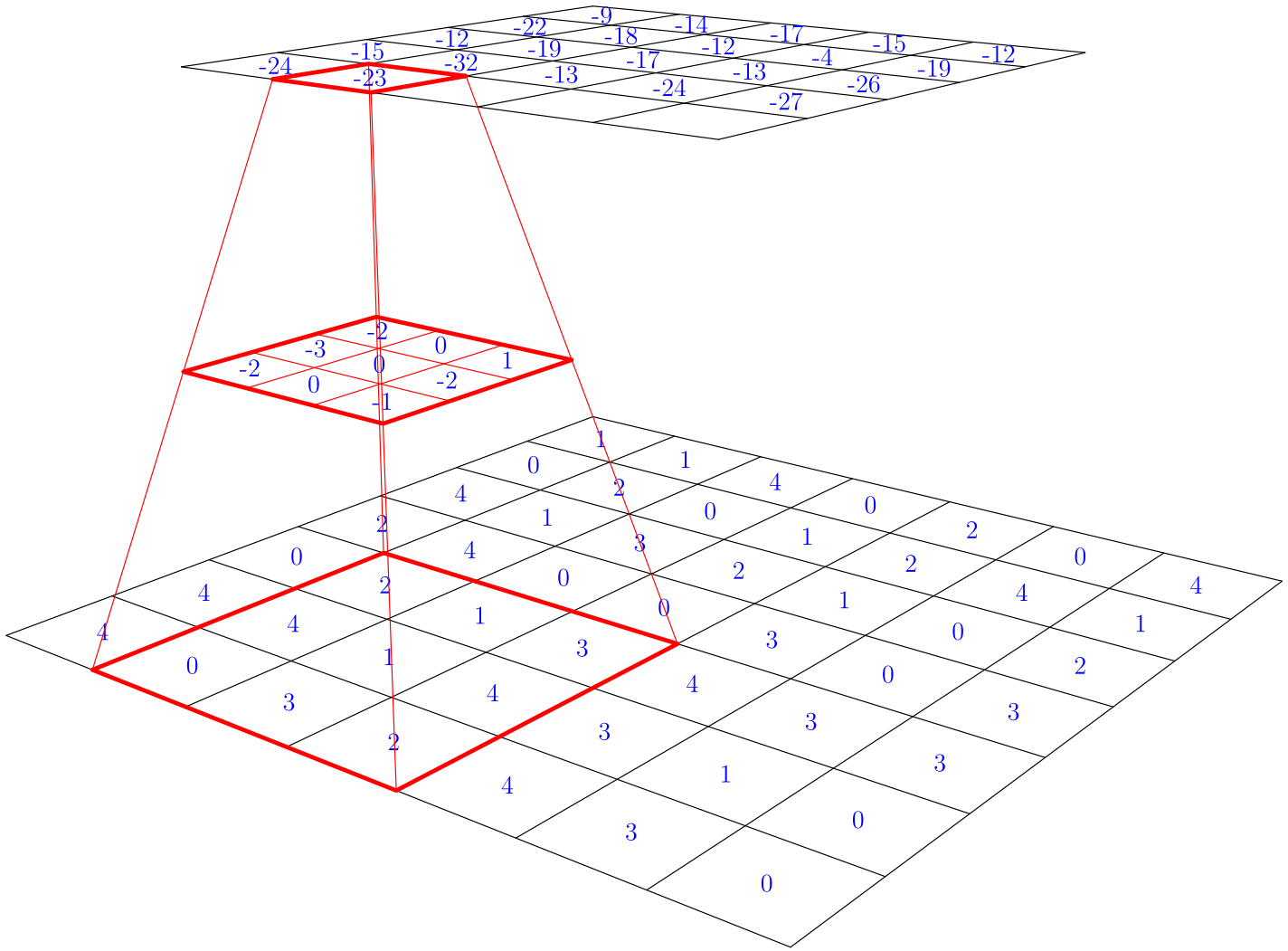
$$-2 \times 3 + 0 \times 0 + 1 \times 3$$
$$-3 \times 4 + 0 \times 3 - 2 \times 3$$
$$-2 \times 3 + 0 \times 1 - 1 \times 0 = -27$$

Convolutional Neural Networks



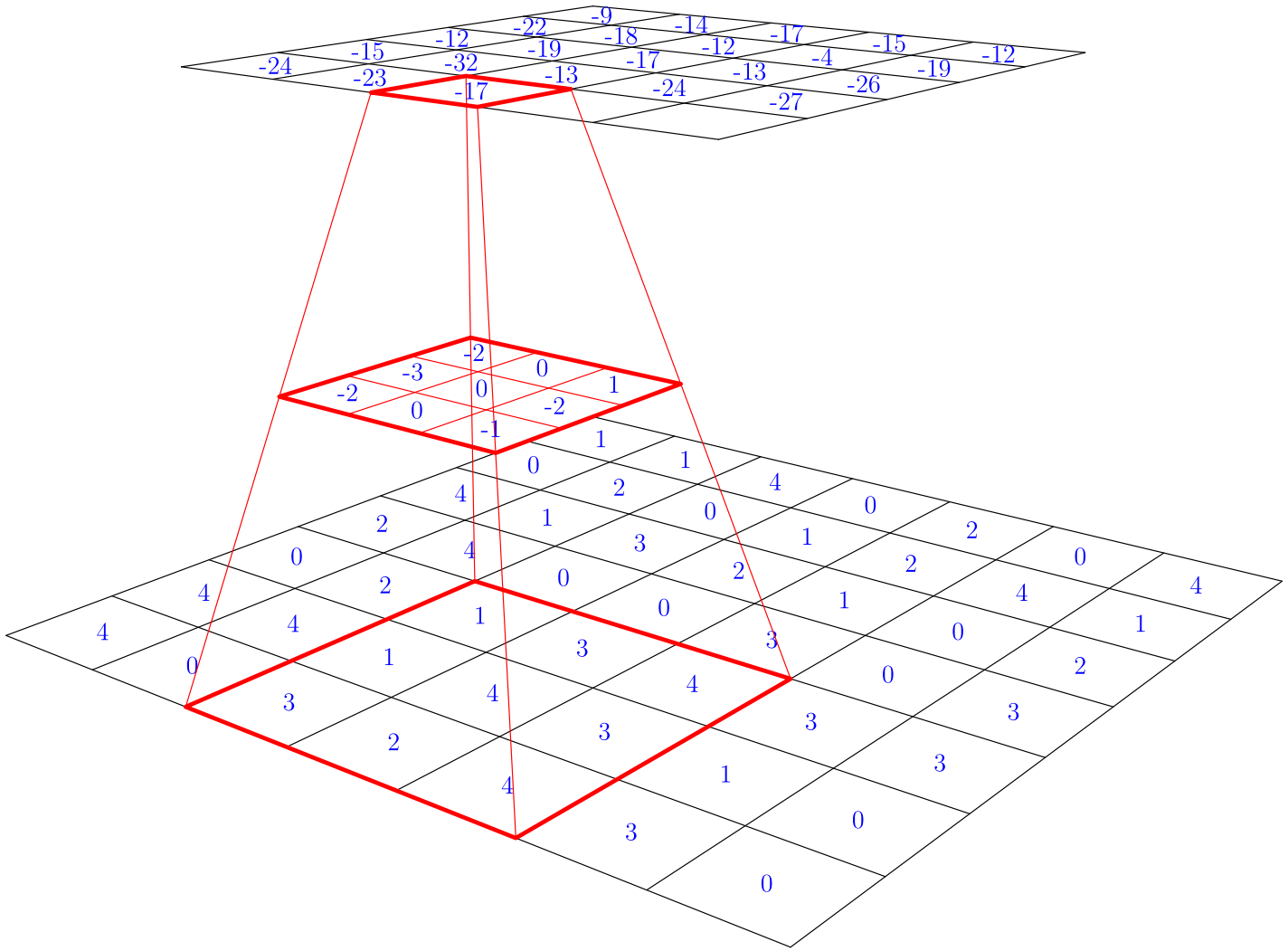
$$-2 \times 0 + 0 \times 2 + 1 \times 1$$
$$-3 \times 4 + 0 \times 4 - 2 \times 1$$
$$-2 \times 4 + 0 \times 0 - 1 \times 3 = -24$$

Convolutional Neural Networks



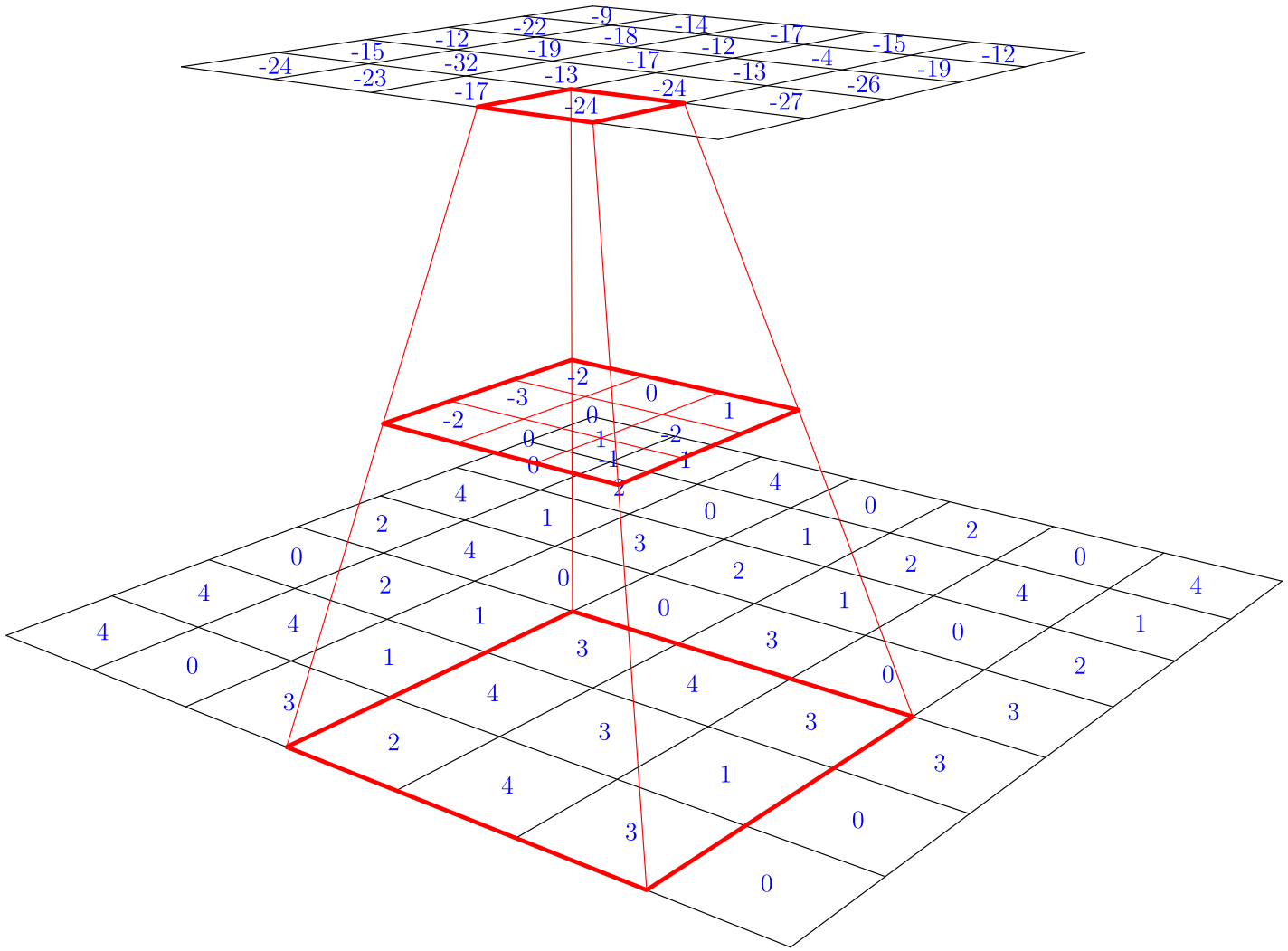
$$-2 \times 2 + 0 \times 1 + 1 \times 3$$
$$-3 \times 4 + 0 \times 1 - 2 \times 4$$
$$-2 \times 0 + 0 \times 3 - 1 \times 2 = -23$$

Convolutional Neural Networks



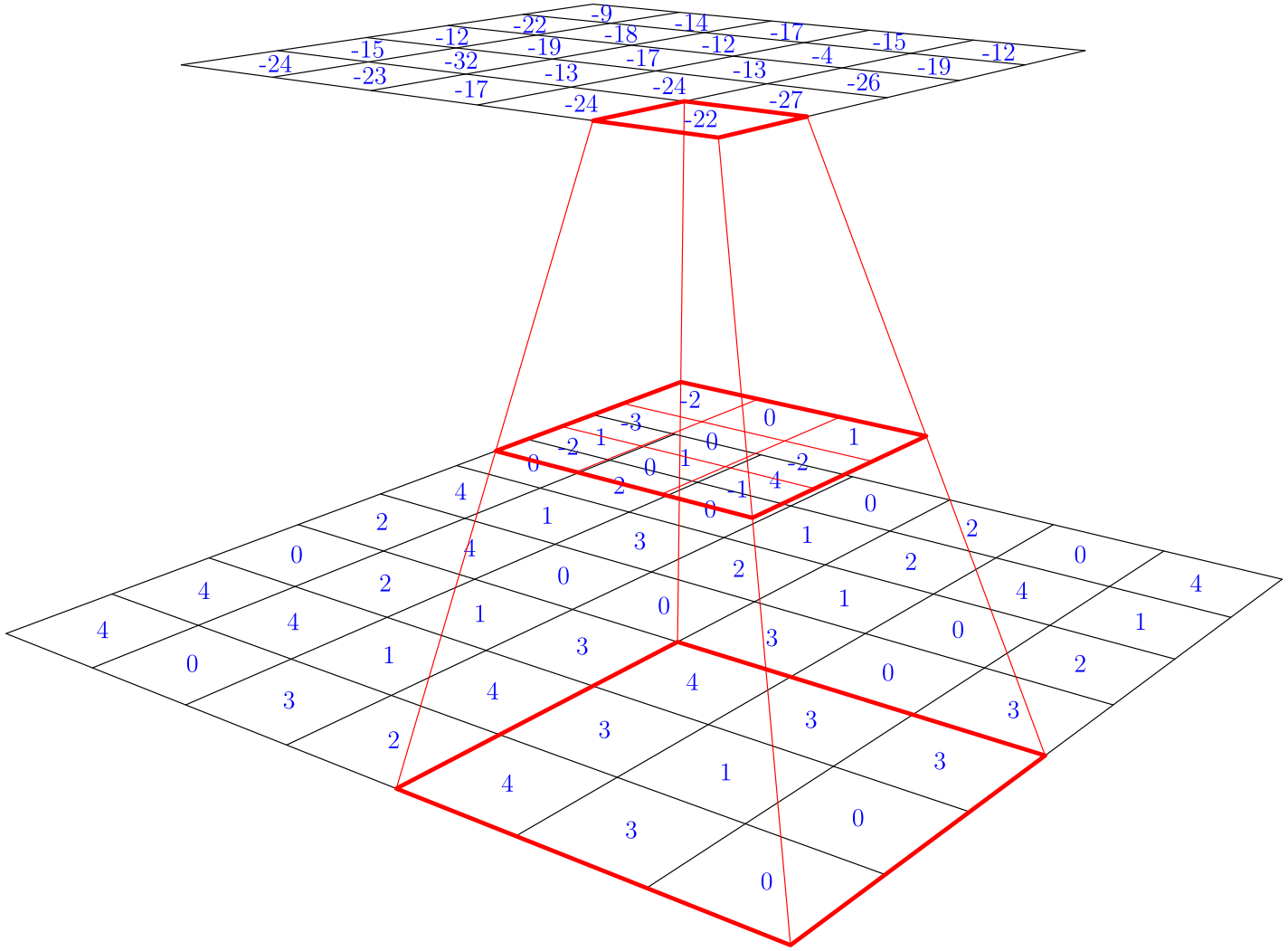
$-2 \times 1 + 0 \times 3 + 1 \times 4$
$-3 \times 1 + 0 \times 4 - 2 \times 3$
$-2 \times 3 + 0 \times 2 - 1 \times 4 = -17$

Convolutional Neural Networks



$$-2 \times 3 + 0 \times 4 + 1 \times 3$$
$$-3 \times 4 + 0 \times 3 - 2 \times 1$$
$$-2 \times 2 + 0 \times 4 - 1 \times 3 = -24$$

Convolutional Neural Networks



$$-2 \times 4 + 0 \times 3 + 1 \times 3$$
$$-3 \times 3 + 0 \times 1 - 2 \times 0$$
$$-2 \times 4 + 0 \times 3 - 1 \times 0 = -22$$

Equivariance

- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years

Equivariance

- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years

Equivariance

- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years

Equivariance

- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years

Equivariance

- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years

Equivariance

- CNNs are not translationally invariant
- The output will be different if we shift the input
- They are (up to edge effects) **equivariant** in that we would get the same result if we shifted both the input and the output in the same way
- They are not perfectly equivariant (they break at the pixel level and at the full image level—due to edges)
- But they are sufficiently good that CNNs have dominated computer vision for years (ignoring vision transformers)

Other Symmetries

- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

Other Symmetries

- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

Other Symmetries

- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant (although there are attempts to remedy this)
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

Other Symmetries

- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant (although there are attempts to remedy this)
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

Other Symmetries

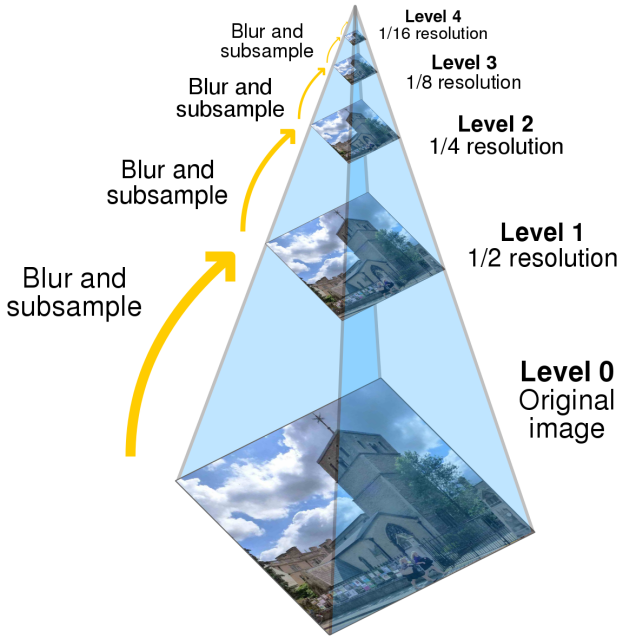
- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant (although there are attempts to remedy this)
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

Other Symmetries

- The identity of an object should not change if we move the camera further away or rotate the camera
- CNNs are not naturally scale or rotationally invariant (although there are attempts to remedy this)
- Rotation invariance doesn't always reflect "the world"—people usually stand on their feet not on their heads
- Although you want rotational invariance when interpreting aerial images

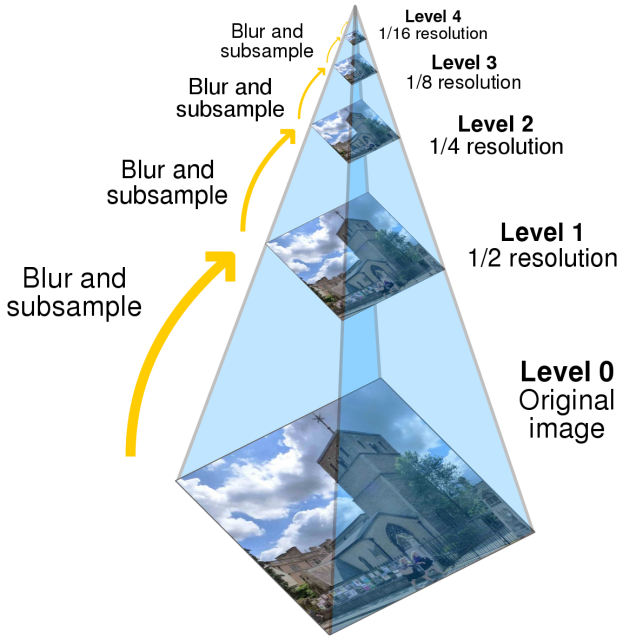
Scale Symmetry

- CNNs are not scale invariant
- There are various pyramid image processing techniques to try and build in approximate scale invariance



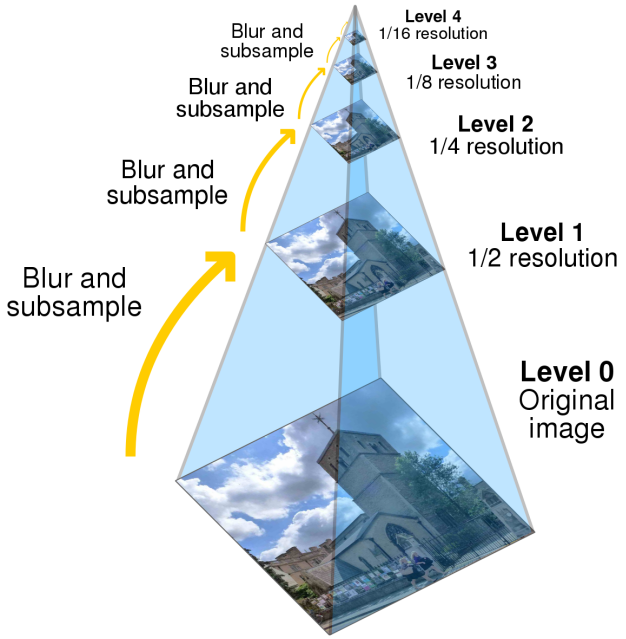
Scale Symmetry

- CNNs are not scale invariant
- There are various pyramid image processing techniques to try and build in approximate scale invariance



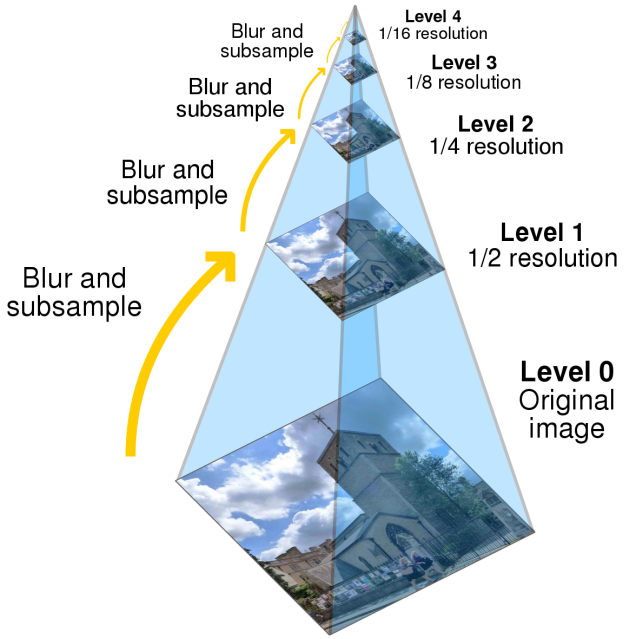
Scale Symmetry

- CNNs are not scale invariant
- There are various pyramid image processing techniques to try and build in approximate scale invariance
- However, in many image classification tasks we concentrate on the object in the foreground



Scale Symmetry

- CNNs are not scale invariant
- There are various pyramid image processing techniques to try and build in approximate scale invariance
- However, in many image classification tasks we concentrate on the object in the foreground
- Many datasets rescale objects in the image so they are roughly the same size



Learning Invariance

- Currently we are not smart enough to come up with networks that are scale invariant or rotational invariant

- We therefore hope that our networks learns this by training on objects of different sizes, orientations, lighting conditions, etc.

- Knowing this is an invariance we want, it is now common practice to do data augmentation where we perform transformations that replicate physical plausible changes in the data

- We are removing spurious rules by (artificially) increasing our dataset size

Learning Invariance

- Currently we are not smart enough to come up with networks that are scale invariant or rotational invariant or invariant to small distortions
- We therefore hope that our networks learns this by training on objects of different sizes, orientations, lighting conditions, etc.
- Knowing this is an invariance we want, it is now common practice to do data augmentation where we perform transformations that replicate physical plausible changes in the data
- We are removing spurious rules by (artificially) increasing our dataset size

Learning Invariance

- Currently we are not smart enough to come up with networks that are scale invariant or rotational invariant or invariant to small distortions
- We therefore hope that our networks learns this by training on objects of different sizes, orientations, lighting conditions, etc.
- Knowing this is an invariance we want, it is now common practice to do data augmentation where we perform transformations that replicate physical plausible changes in the data
- We are removing spurious rules by (artificially) increasing our dataset size

Learning Invariance

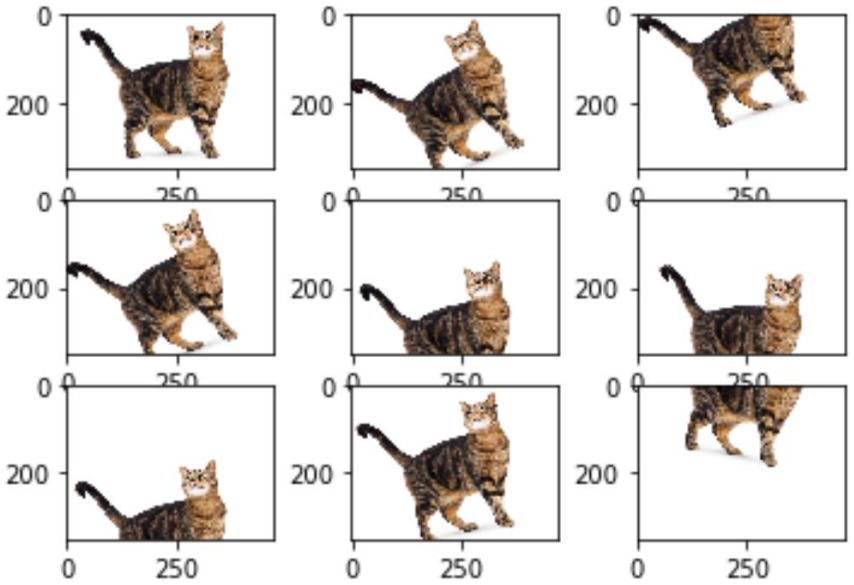
- Currently we are not smart enough to come up with networks that are scale invariant or rotational invariant or invariant to small distortions
- We therefore hope that our networks learns this by training on objects of different sizes, orientations, lighting conditions, etc.
- Knowing this is an invariance we want, it is now common practice to do data augmentation where we perform transformations that replicate physical plausible changes in the data
- We are removing spurious rules by (artificially) increasing our dataset size

Learning Invariance

- Currently we are not smart enough to come up with networks that are scale invariant or rotational invariant or invariant to small distortions
- We therefore hope that our networks learns this by training on objects of different sizes, orientations, lighting conditions, etc.
- Knowing this is an invariance we want, it is now common practice to do data augmentation where we perform transformations that replicate physical plausible changes in the data
- We are removing spurious rules by (artificially) increasing our dataset size

Augmentations

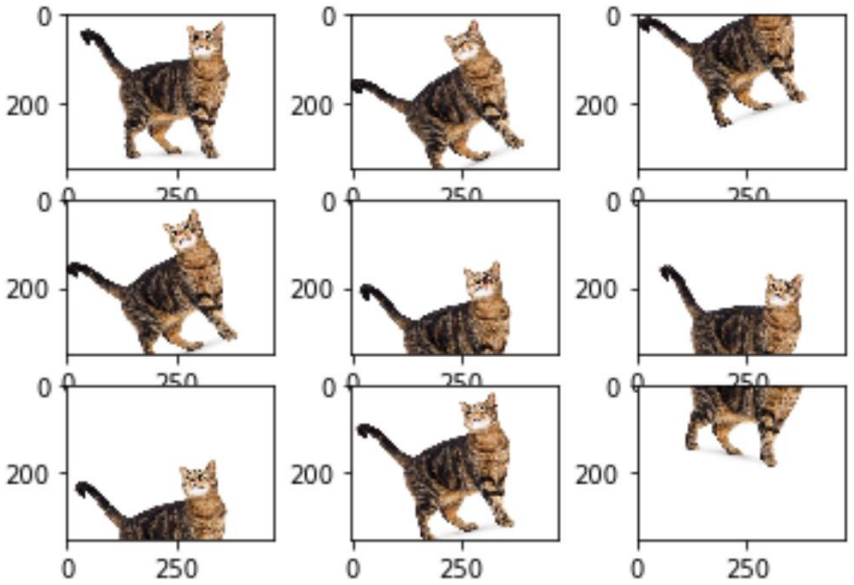




- Data augmentation is standard practice in deep learning for images

Augmentations

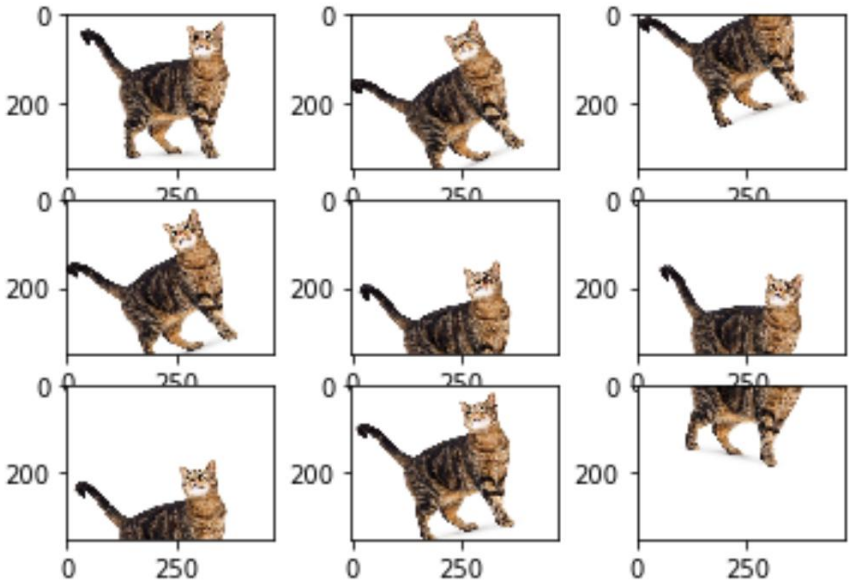




- Data augmentation is standard practice in deep learning for images

Augmentations





- Data augmentation is standard practice in deep learning for images and makes a very considerable difference

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering
- We might do this by normalising the output

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering
- We might do this by normalising the output

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output—use cosine similarity rather than distance

$$\cos(\theta) = \frac{\boldsymbol{a}^\mathsf{T}\boldsymbol{b}}{\|\boldsymbol{a}\|\|\boldsymbol{b}\|}$$

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output—use cosine similarity rather than distance

$$\cos(\theta) = \frac{\boldsymbol{a}^\mathsf{T}\boldsymbol{b}}{\|\boldsymbol{a}\|\|\boldsymbol{b}\|}$$

- These make the results invariant

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output—use cosine similarity rather than distance

$$\cos(\theta) = \frac{\boldsymbol{a}^\mathsf{T}\boldsymbol{b}}{\|\boldsymbol{a}\|\|\boldsymbol{b}\|}$$

- These make the results invariant to the distance of the camera

Normalisation

- Sometimes clever normalisation can make a lot of difference to the performance of an algorithm
- We might do this by feature engineering (divide the length on the nose by the width between the persons eyes)
- We might do this by normalising the output—use cosine similarity rather than distance

$$\cos(\theta) = \frac{\boldsymbol{a}^\mathsf{T}\boldsymbol{b}}{\|\boldsymbol{a}\|\|\boldsymbol{b}\|}$$

- These make the results invariant to the distance of the camera or the length of $\boldsymbol{a}$ and $\boldsymbol{b}$

Outline

1. Inductive Bias
2. Invariance
3. **Group Theory**
4. Symmetry in Machine Learning



Group Theory

- Symmetries are very well studied in mathematics
- The language of symmetries are **groups**
- These capture the structure of transformations that represent some symmetry
- Group theory is highly abstract and it is easy to lose sight of its connection to symmetry, but it has proved itself extraordinarily powerful in revealing structures

Group Theory

- Symmetries are very well studied in mathematics
- The language of symmetries are **groups**
- These capture the structure of transformations that represent some symmetry
- Group theory is highly abstract and it is easy to lose sight of its connection to symmetry, but it has proved itself extraordinarily powerful in revealing structures

Group Theory

- Symmetries are very well studied in mathematics
- The language of symmetries are **groups**
- These capture the structure of transformations that represent some symmetry
- Group theory is highly abstract and it is easy to lose sight of its connection to symmetry, but it has proved itself extraordinarily powerful in revealing structures

Group Theory

- Symmetries are very well studied in mathematics
- The language of symmetries are **groups**
- These capture the structure of transformations that represent some symmetry
- Group theory is highly abstract and it is easy to lose sight of its connection to symmetry, but it has proved itself extraordinarily powerful in revealing structures

Groups

- A group consists of a set, $\mathcal{G} = \{a, b, \cdots\}$, whose elements we can think of as transformations
- And an operator "$\cdot$", usually composition: $a \cdot b$ means apply transformation b and then a
- The set and operation form a group if they satisfy four axioms
  - ⋆ closure: for all $a, b \in \mathcal{G}$ then $a \cdot b \in \mathcal{G}$
  - ⋆ associativity: $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
  - ⋆ existence of an identity e such that $a \cdot e = e \cdot a = a$ for all a
  - ⋆ Every element, a, has an inverse a^{-1} such that $a^{-1} \cdot a = a \cdot a^{-1} = e$

Groups

- A group consists of a set, $\mathcal{G} = \{a, b, \cdots\}$, whose elements we can think of as transformations
- And an operator "$\cdot$", usually composition: $a \cdot b$ means apply transformation b and then a
- The set and operation form a group if they satisfy four axioms
  - ⋆ closure: for all $a, b \in \mathcal{G}$ then $a \cdot b \in \mathcal{G}$
  - ⋆ associativity: $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
  - ⋆ existence of an identity e such that $a \cdot e = e \cdot a = a$ for all a
  - ⋆ Every element, a, has an inverse a^{-1} such that $a^{-1} \cdot a = a \cdot a^{-1} = e$

Groups

- A group consists of a set, $\mathcal{G} = \{a, b, \cdots\}$, whose elements we can think of as transformations
- And an operator "$\cdot$", usually composition: $a \cdot b$ means apply transformation b and then a
- The set and operation form a group if they satisfy four axioms
  - ⋆ closure: for all $a, b \in \mathcal{G}$ then $a \cdot b \in \mathcal{G}$
  - ⋆ associativity: $a \cdot (b \cdot c) = (a \cdot b) \cdot c$
  - ⋆ existence of an identity e such that $a \cdot e = e \cdot a = a$ for all a
  - ⋆ Every element, a, has an inverse a^{-1} such that $a^{-1} \cdot a = a \cdot a^{-1} = e$

Tossing a coin

- We consider a coin with two actions
  - ⋆ flip the coin: f
  - ⋆ leave the coin alone: e
- These form a group (the cyclic group C_2)
  - ⋆ Closure is obvious $f \cdot e = f$, $f \cdot f = e$, $e \cdot e = e$ and $e \cdot f = f$
  - ⋆ Associativity: e.g. $(e \cdot f) \cdot f = e \cdot (f \cdot f)$
  - ⋆ e is the identity
  - ⋆ $e^{-1} = e$ and $f^{-1} = f$

Tossing a coin

- We consider a coin with two actions
  - ⋆ flip the coin: f
  - ⋆ leave the coin alone: e
- These form a group (the cyclic group C_2)
  - ⋆ Closure is obvious $f \cdot e = f$, $f \cdot f = e$, $e \cdot e = e$ and $e \cdot f = f$
  - ⋆ Associativity: e.g. $(e \cdot f) \cdot f = e \cdot (f \cdot f)$
  - ⋆ e is the identity
  - ⋆ $e^{-1} = e$ and $f^{-1} = f$

The Four Group

- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant

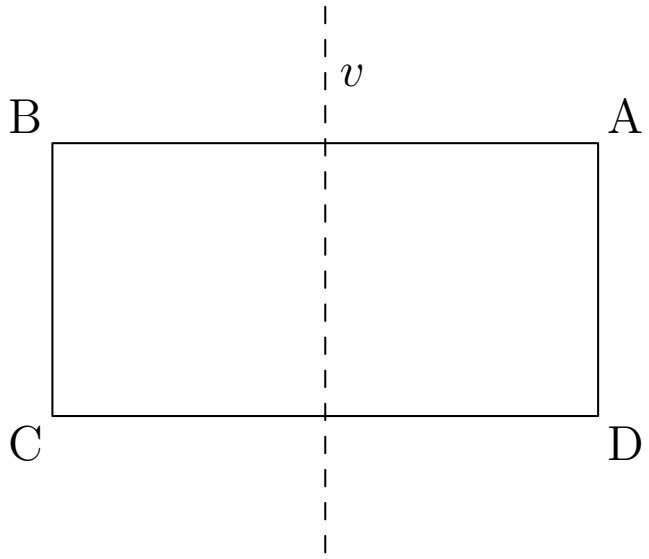
The Four Group

- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant



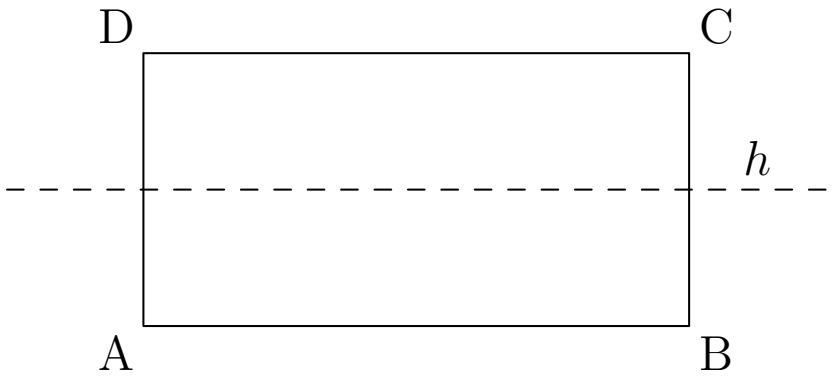
The Four Group

- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant



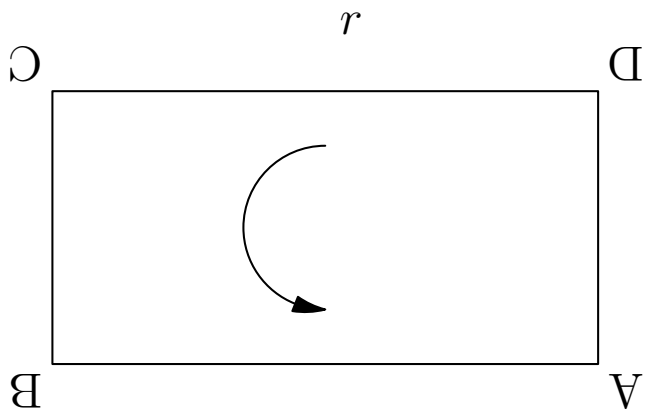
The Four Group

- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant



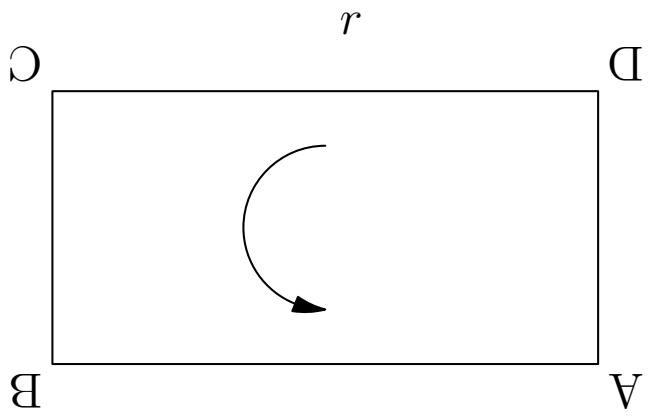
The Four Group

- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant



The Four Group

- A slightly less trivial example represents the set of transformations that leaves a rectangle invariant



	e	v	h	r
e	e	v	h	r
v	v	e	r	h
h	h	r	e	v
r	r	h	v	e

Non-commutativity

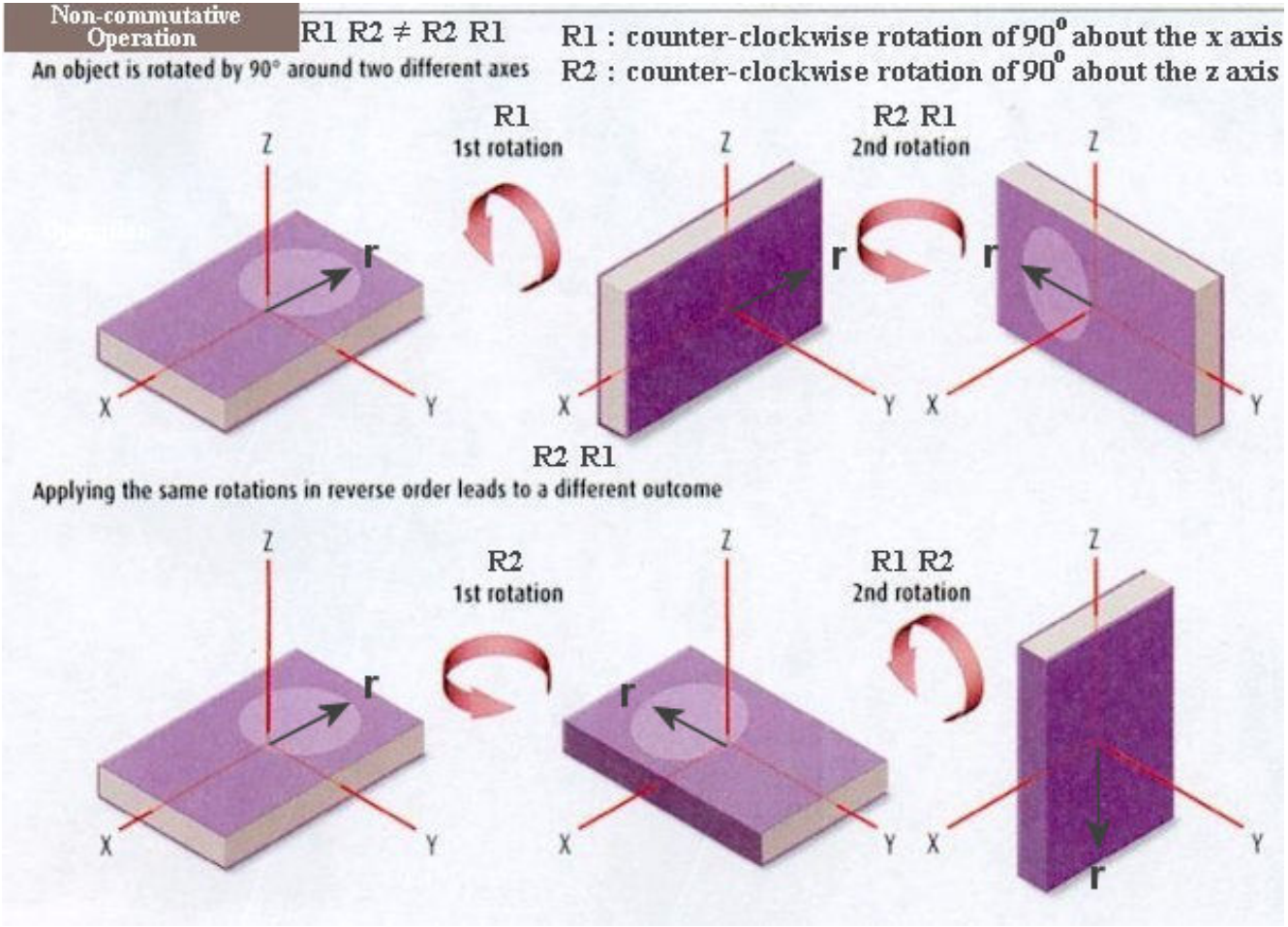
- Not all groups commute
- For 3-d rotations : $R_x \cdot R_z \neq R_z \cdot R_x$

Non-commutativity

- Not all groups commute
- For 3-d rotations : $R_x \cdot R_z \neq R_z \cdot R_x$

Non-commutativity

- Not all groups commute
- For 3-d rotations : $R_x \cdot R_z \neq R_z \cdot R_x$



Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements

- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements

- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements

- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements

- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements

- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements (you can reorder elements in n! ways)
- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Permutations

- The set of permutations of n objects form a group (known as the symmetric group S_n)
- The composition of a permutation is a permutation
- There is an identity (do nothing)
- For every permutation there is an inverse permutation
- Obviously S_n has $n!$ elements (you can reorder elements in n! ways)
- Permutations have a lot of structure (in terms of elements cycling, subgroups, etc.)

Infinite and Continuous Groups

- The integers under addition form a group (identity 0, inverse $-n$): the discrete shifts
- The reals under addition describe translations; the positive reals under multiplication describe scalings
- Smooth continuous groups are called **Lie groups**: rotations, translations, the symmetries of relativity
- They can be represented by matrices, and are studied through their infinitesimal transformations (Lie algebras)
- Much used in physics, and increasingly in machine learning

Infinite and Continuous Groups

- The integers under addition form a group (identity 0, inverse $-n$): the discrete shifts
- The reals under addition describe translations; the positive reals under multiplication describe scalings
- Smooth continuous groups are called **Lie groups**: rotations, translations, the symmetries of relativity
- They can be represented by matrices, and are studied through their infinitesimal transformations (Lie algebras)
- Much used in physics, and increasingly in machine learning

Infinite and Continuous Groups

- The integers under addition form a group (identity 0, inverse $-n$): the discrete shifts
- The reals under addition describe translations; the positive reals under multiplication describe scalings
- Smooth continuous groups are called **Lie groups**: rotations, translations, the symmetries of relativity
- They can be represented by matrices, and are studied through their infinitesimal transformations (Lie algebras)
- Much used in physics, and increasingly in machine learning

Infinite and Continuous Groups

- The integers under addition form a group (identity 0, inverse $-n$): the discrete shifts
- The reals under addition describe translations; the positive reals under multiplication describe scalings
- Smooth continuous groups are called **Lie groups**: rotations, translations, the symmetries of relativity
- They can be represented by matrices, and are studied through their infinitesimal transformations (Lie algebras)
- Much used in physics, and increasingly in machine learning

Infinite and Continuous Groups

- The integers under addition form a group (identity 0, inverse $-n$): the discrete shifts
- The reals under addition describe translations; the positive reals under multiplication describe scalings
- Smooth continuous groups are called **Lie groups**: rotations, translations, the symmetries of relativity
- They can be represented by matrices, and are studied through their infinitesimal transformations (Lie algebras)
- Much used in physics, and increasingly in machine learning

Rotations $SO(n)$

- The set of $n \times n$ orthogonal matrices (matrices, $\mathbf{M}$, such that $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\mathbf{M}^\mathsf{T} = \mathbf{I}$) form a group $O(n)$
- Note that $\det(\mathbf{M}) = \pm 1$
- These correspond to rotations and reflections
- The group of matrices that don't correspond to reflections (where $\det(\mathbf{M}) = 1$) form a subgroup $SO(n)$
- $SO(2)$ consist of the 2-d rotation matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Rotations $SO(n)$

- The set of $n \times n$ orthogonal matrices (matrices, $\mathbf{M}$, such that $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\mathbf{M}^\mathsf{T} = \mathbf{I}$) form a group $O(n)$
- Note that $\det(\mathbf{M}) = \pm 1$
- These correspond to rotations and reflections
- The group of matrices that don't correspond to reflections (where $\det(\mathbf{M}) = 1$) form a subgroup $SO(n)$
- $SO(2)$ consist of the 2-d rotation matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Rotations $SO(n)$

- The set of $n \times n$ orthogonal matrices (matrices, $\mathbf{M}$, such that $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\mathbf{M}^\mathsf{T} = \mathbf{I}$) form a group $O(n)$
- Note that $\det(\mathbf{M}) = \pm 1$
- These correspond to rotations and reflections
- The group of matrices that don't correspond to reflections (where $\det(\mathbf{M}) = 1$) form a subgroup $SO(n)$
- $SO(2)$ consist of the 2-d rotation matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Rotations $SO(n)$

- The set of $n \times n$ orthogonal matrices (matrices, $\mathbf{M}$, such that $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\mathbf{M}^\mathsf{T} = \mathbf{I}$) form a group $O(n)$
- Note that $\det(\mathbf{M}) = \pm 1$
- These correspond to rotations and reflections
- The group of matrices that don't correspond to reflections (where $\det(\mathbf{M}) = 1$) form a subgroup $SO(n)$
- $SO(2)$ consist of the 2-d rotation matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Rotations $SO(n)$

- The set of $n \times n$ orthogonal matrices (matrices, $\mathbf{M}$, such that $\mathbf{M}^\mathsf{T}\mathbf{M} = \mathbf{M}\mathbf{M}^\mathsf{T} = \mathbf{I}$) form a group $O(n)$
- Note that $\det(\mathbf{M}) = \pm 1$
- These correspond to rotations and reflections
- The group of matrices that don't correspond to reflections (where $\det(\mathbf{M}) = 1$) form a subgroup $SO(n)$
- $SO(2)$ consist of the 2-d rotation matrices

$$\begin{pmatrix} \cos(\theta) & \sin(\theta) \\ -\sin(\theta) & \cos(\theta) \end{pmatrix}$$

Outline

1. Inductive Bias
2. Invariance
3. Group Theory
4. **Symmetry in Machine Learning**



Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot \boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g' \in G} f(g' \cdot \boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g \cdot h$ just reorders the group (closure and inverses)

- So $\bar{f}$ is invariant
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot \boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g' \in G} f(g' \cdot \boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g \cdot h$ just reorders the group (closure and inverses)

- So $\bar{f}$ is invariant
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot\boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g'\in G} f(g'\cdot\boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g\cdot h$ just reorders the group (closure and inverses)

- So $\bar{f}$ is invariant
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot\boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g\in G} f(g\cdot h\cdot\boldsymbol{x}) = \frac{1}{|G|}\sum_{g'\in G} f(g'\cdot\boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g\cdot h$ just reorders the group (closure and inverses)

- So $\bar{f}$ is invariant
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot \boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g' \in G} f(g' \cdot \boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g \cdot h$ just reorders the group (closure and inverses)

- So $\bar{f}$ is invariant—here the group axioms earn their keep
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Invariance by Averaging

- Given any function f and a finite group G of transformations, define

$$\bar{f}(\boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot \boldsymbol{x})$$

- For any $h \in G$

$$\bar{f}(h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g \in G} f(g \cdot h \cdot \boldsymbol{x}) = \frac{1}{|G|} \sum_{g' \in G} f(g' \cdot \boldsymbol{x}) = \bar{f}(\boldsymbol{x})$$

  because $g \mapsto g \cdot h$ just reorders the group (closure and inverses)

- So $\bar{f}$ is invariant—here the group axioms earn their keep
- Data augmentation approximates this in training; **test-time augmentation** does it exactly

Equivariance and Convolution

- A map is **equivariant** if transforming the input transforms the output in the same way: $f(g \cdot \boldsymbol{x}) = g \cdot f(\boldsymbol{x})$
- Convolution is shift-equivariant: shifting then convolving is the same as convolving then shifting
- In fact the linear maps that commute with all (circular) shifts are exactly the convolutions
- A 3×3 filter has 9 weights, against n^4 for a dense layer on an $n \times n$ image
- Building in the symmetry removes parameters, and with them variance (bias-variance lecture)

Equivariance and Convolution

- A map is **equivariant** if transforming the input transforms the output in the same way: $f(g \cdot \boldsymbol{x}) = g \cdot f(\boldsymbol{x})$
- Convolution is shift-equivariant: shifting then convolving is the same as convolving then shifting
- In fact the linear maps that commute with all (circular) shifts are exactly the convolutions
- A 3×3 filter has 9 weights, against n^4 for a dense layer on an $n \times n$ image
- Building in the symmetry removes parameters, and with them variance (bias-variance lecture)

Equivariance and Convolution

- A map is **equivariant** if transforming the input transforms the output in the same way: $f(g \cdot \boldsymbol{x}) = g \cdot f(\boldsymbol{x})$
- Convolution is shift-equivariant: shifting then convolving is the same as convolving then shifting
- In fact the linear maps that commute with all (circular) shifts are exactly the convolutions
- A 3×3 filter has 9 weights, against n^4 for a dense layer on an $n \times n$ image
- Building in the symmetry removes parameters, and with them variance (bias-variance lecture)

Equivariance and Convolution

- A map is **equivariant** if transforming the input transforms the output in the same way: $f(g \cdot \boldsymbol{x}) = g \cdot f(\boldsymbol{x})$
- Convolution is shift-equivariant: shifting then convolving is the same as convolving then shifting
- In fact the linear maps that commute with all (circular) shifts are exactly the convolutions
- A 3×3 filter has 9 weights, against n^4 for a dense layer on an $n \times n$ image
- Building in the symmetry removes parameters, and with them variance (bias-variance lecture)

Equivariance and Convolution

- A map is **equivariant** if transforming the input transforms the output in the same way: $f(g \cdot \boldsymbol{x}) = g \cdot f(\boldsymbol{x})$
- Convolution is shift-equivariant: shifting then convolving is the same as convolving then shifting
- In fact the linear maps that commute with all (circular) shifts are exactly the convolutions
- A 3×3 filter has 9 weights, against n^4 for a dense layer on an $n \times n$ image
- Building in the symmetry removes parameters, and with them variance (bias-variance lecture)

Permutation Symmetry

- Often the input is a **set**: points in a cloud, objects in a scene, atoms in a molecule
- Any function of the form

$$f(\{\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n\}) = \rho\Big(\sum_{i=1}^{n}\boldsymbol{\phi}(\boldsymbol{x}_i)\Big)$$

  is invariant to the order (**Deep Sets**)
- Graph neural networks update each node from a sum over its neighbours: equivariant to relabelling the nodes
- The symmetric group S_n is doing the work

Permutation Symmetry

- Often the input is a **set**: points in a cloud, objects in a scene, atoms in a molecule
- Any function of the form

$$f(\{\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n\}) = \rho\Big(\sum_{i=1}^{n}\boldsymbol{\phi}(\boldsymbol{x}_i)\Big)$$

  is invariant to the order (**Deep Sets**)

- Graph neural networks update each node from a sum over its neighbours: equivariant to relabelling the nodes
- The symmetric group S_n is doing the work

Permutation Symmetry

- Often the input is a **set**: points in a cloud, objects in a scene, atoms in a molecule
- Any function of the form

$$f(\{\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n\}) = \rho\Big(\sum_{i=1}^{n}\boldsymbol{\phi}(\boldsymbol{x}_i)\Big)$$

  is invariant to the order (**Deep Sets**)
- Graph neural networks update each node from a sum over its neighbours: equivariant to relabelling the nodes
- The symmetric group S_n is doing the work

Permutation Symmetry

- Often the input is a **set**: points in a cloud, objects in a scene, atoms in a molecule
- Any function of the form

$$f(\{\boldsymbol{x}_1,\ldots,\boldsymbol{x}_n\}) = \rho\Big(\sum_{i=1}^{n}\boldsymbol{\phi}(\boldsymbol{x}_i)\Big)$$

  is invariant to the order (**Deep Sets**)
- Graph neural networks update each node from a sum over its neighbours: equivariant to relabelling the nodes
- The symmetric group S_n is doing the work

Transformers

- Self-attention: each token i computes a query $\boldsymbol{q}_i$, key $\boldsymbol{k}_i$ and value $\boldsymbol{v}_i$, and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d}}\right)\boldsymbol{v}_j$$

- Permuting the tokens just permutes the outputs: self-attention is permutation-equivariant

- So on its own it cannot see word order

- Knowing the symmetry tells you what the model cannot know

Transformers

- Self-attention: each token i computes a query $\boldsymbol{q}_i$, key $\boldsymbol{k}_i$ and value $\boldsymbol{v}_i$, and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d}}\right)\boldsymbol{v}_j$$

- Permuting the tokens just permutes the outputs: self-attention is permutation-equivariant

- So on its own it cannot see word order

- Knowing the symmetry tells you what the model cannot know

Transformers

- Self-attention: each token i computes a query $\boldsymbol{q}_i$, key $\boldsymbol{k}_i$ and value $\boldsymbol{v}_i$, and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d}}\right)\boldsymbol{v}_j$$

- Permuting the tokens just permutes the outputs: self-attention is permutation-equivariant

- So on its own it cannot see word order

- Knowing the symmetry tells you what the model cannot know

Transformers

- Self-attention: each token i computes a query $\boldsymbol{q}_i$, key $\boldsymbol{k}_i$ and value $\boldsymbol{v}_i$, and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d}}\right)\boldsymbol{v}_j$$

- Permuting the tokens just permutes the outputs: self-attention is permutation-equivariant

- So on its own it cannot see word order—transformers add **positional encodings** to break this symmetry

- Knowing the symmetry tells you what the model cannot know

Transformers

- Self-attention: each token i computes a query $\boldsymbol{q}_i$, key $\boldsymbol{k}_i$ and value $\boldsymbol{v}_i$, and outputs

$$\boldsymbol{y}_i = \sum_j \mathrm{softmax}_j\left(\frac{\boldsymbol{q}_i^\top \boldsymbol{k}_j}{\sqrt{d}}\right)\boldsymbol{v}_j$$

- Permuting the tokens just permutes the outputs: self-attention is permutation-equivariant

- So on its own it cannot see word order—transformers add **positional encodings** to break this symmetry

- Knowing the symmetry tells you what the model cannot know

Rotations and Molecules

- The energy of a molecule does not change if we rotate or move it: it is invariant to $SO(3)$ and translations
- The forces on the atoms rotate with the molecule: they are equivariant
- Equivariant networks build this in, and are now standard for learning about molecules and materials
- AlphaFold 2's structure module is built to respect rotations and translations of the protein
- This area is called **geometric deep learning**

Rotations and Molecules

- The energy of a molecule does not change if we rotate or move it: it is invariant to $SO(3)$ and translations
- The forces on the atoms rotate with the molecule: they are equivariant
- Equivariant networks build this in, and are now standard for learning about molecules and materials
- AlphaFold 2's structure module is built to respect rotations and translations of the protein
- This area is called **geometric deep learning**

Rotations and Molecules

- The energy of a molecule does not change if we rotate or move it: it is invariant to $SO(3)$ and translations
- The forces on the atoms rotate with the molecule: they are equivariant
- Equivariant networks build this in, and are now standard for learning about molecules and materials
- AlphaFold 2's structure module is built to respect rotations and translations of the protein
- This area is called **geometric deep learning**

Rotations and Molecules

- The energy of a molecule does not change if we rotate or move it: it is invariant to $SO(3)$ and translations
- The forces on the atoms rotate with the molecule: they are equivariant
- Equivariant networks build this in, and are now standard for learning about molecules and materials
- AlphaFold 2's structure module is built to respect rotations and translations of the protein
- This area is called **geometric deep learning**

Rotations and Molecules

- The energy of a molecule does not change if we rotate or move it: it is invariant to $SO(3)$ and translations
- The forces on the atoms rotate with the molecule: they are equivariant
- Equivariant networks build this in, and are now standard for learning about molecules and materials
- AlphaFold 2's structure module is built to respect rotations and translations of the protein
- This area is called **geometric deep learning**

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated—reducing the chances of finding spurious rules
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated—reducing the chances of finding spurious rules
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated—reducing the chances of finding spurious rules
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated—reducing the chances of finding spurious rules
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks

Summary

- Invariances are important as if a machine properly captures a true invariance it can massively limit the space of functions being approximated—reducing the chances of finding spurious rules
- Often it is hard to build-in invariances into the machine learning model and practitioners will often resort to data augmentation in the hope that these invariances can be learned
- There is a mathematical language around describing the structure of symmetries: **group theory**
- Groups explain why averaging gives invariance, and underlie CNNs, graph networks, transformers and equivariant networks
- I'm not going to expect you to know any details about group theory

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

The Course in One Slide

Represent	vectors, inner products, eigenvectors, SVD, PCA
Fit	derivatives, gradient descent, Newton, SGD, Adam
Constrain	Lagrange, convexity, duality, SVMs, kernels
Be uncertain	probability, Bayes, Gaussians, GPs, MCMC
Compress	entropy, information, KL, optimal transport
Generalise	bias and variance, regularisation, symmetry

- These ideas recur: the SVD in PCA, ridge and double descent; Lagrange multipliers in SVMs, maximum entropy and optimal transport

Final Thoughts

- Machine learning works when the model's structure matches the structure of the world
- Priors, regularisers and symmetries are three ways of saying what that structure is
- Mathematics is the language for saying it precisely
- The methods will keep changing; the mathematics underneath them changes far more slowly

Final Thoughts

- Machine learning works when the model's structure matches the structure of the world
- Priors, regularisers and symmetries are three ways of saying what that structure is
- Mathematics is the language for saying it precisely
- The methods will keep changing; the mathematics underneath them changes far more slowly

Final Thoughts

- Machine learning works when the model's structure matches the structure of the world
- Priors, regularisers and symmetries are three ways of saying what that structure is
- Mathematics is the language for saying it precisely
- The methods will keep changing; the mathematics underneath them changes far more slowly

Final Thoughts

- Machine learning works when the model's structure matches the structure of the world
- Priors, regularisers and symmetries are three ways of saying what that structure is
- Mathematics is the language for saying it precisely
- The methods will keep changing; the mathematics underneath them changes far more slowly